

Clase Visión Artificial

2026-08-27 22:04 UTC

Recorded by

Jorge Alejandro Silvestri

Organized by

Alesio Esteban Sinopoli

Channel

General



MILITELLO NICOLAS URIEL ✂



LOPEZ FERME NAHUEL EZEQUIEL ✂



Jorge Alejandro Silvestri



DI STILIO GASTON ✂



PAGNOTTA PABLO



MATEO FEDERICO ✂



BAINI HERNAN JAVIER ✂



MAGNATTA EZEQUIEL DAMIAN ✂



SAINI LUIS ALBERTO ✂





Imágenes binarias y umbrales



Archivo Editar Ver Insertar Formato Diapositiva Estructura ...



Presentación de diapositivas



Actualizar



Ajustar



Tr



Fondo

Diseño

Tema

Transición



1



2



3



4



Imágenes binarias y umbrales



Mejorar esta diapositiva



Diapositiva



Contenido multimedia



Cargas



Grabar



Transformar



Revisión general 2026.
Basada en el original de 2020.



Imágenes binarias y umbrales

Visión artificial

Alejandro Silvestri



Qué es una imagen binaria

Definición y Representación

Es aquella cuyos píxeles tienen únicamente valores lógicos **true** o **false**.

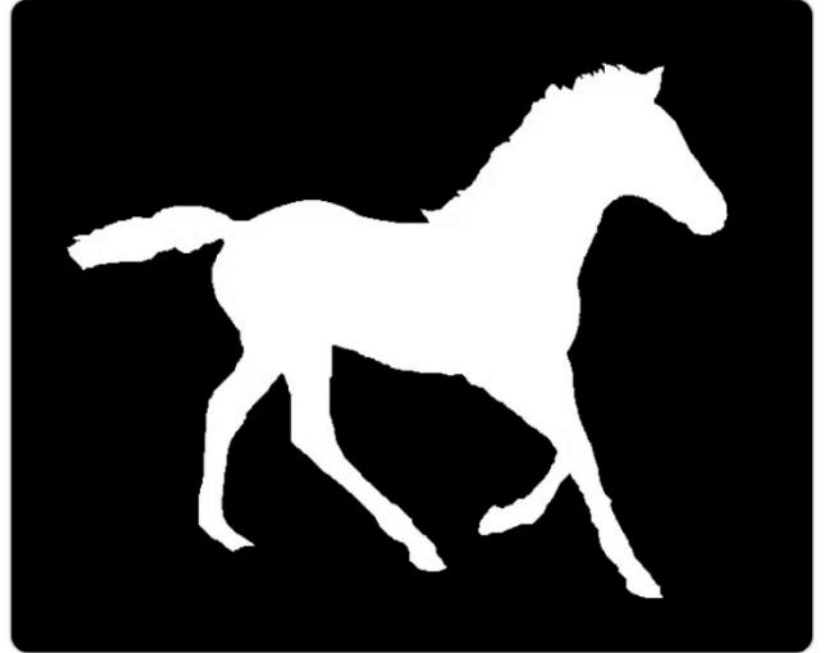
Interpretación en OpenCV (escala de grises de 256 niveles):

- **0** se interpreta como **false**
- **Distinto de 0** se interpreta como **true**

Visualización Estándar

Para visualizar correctamente la imagen binaria, se suelen normalizar los valores utilizando los extremos del rango:

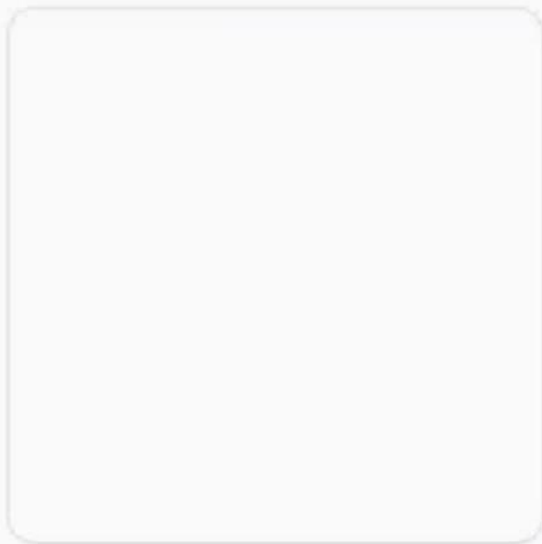
- **Valor 0** representa el color **Negro** (false)
- **Valor 255** representa el color **Blanco** (true)



Ejemplo de segmentación binaria (Silueta en blanco/true sobre fondo negro/false)

Máscaras

1. Imagen Original



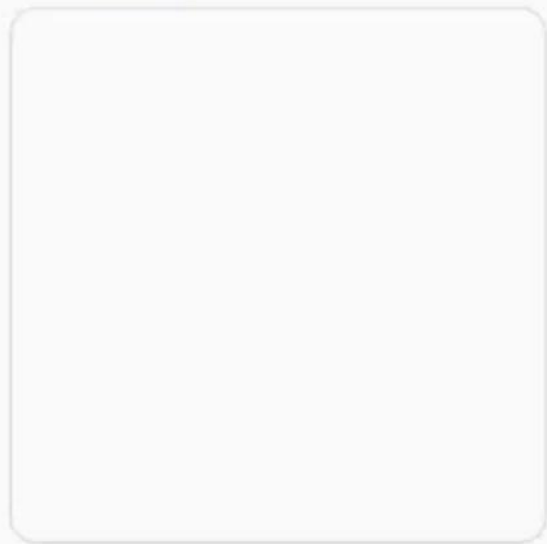
La imagen original sobre la cual queremos aislar o destacar un objeto de interés.

2. Máscara Binaria



Imagen binaria donde los píxeles blancos (true) definen el área que se conservará.

3. Resultado Enmascarado



Resultado tras copiar únicamente los píxeles correspondientes a la máscara.

Máscaras

1. Imagen Original



La imagen original sobre la cual queremos aislar o destacar un objeto de interés.

2. Máscara Binaria

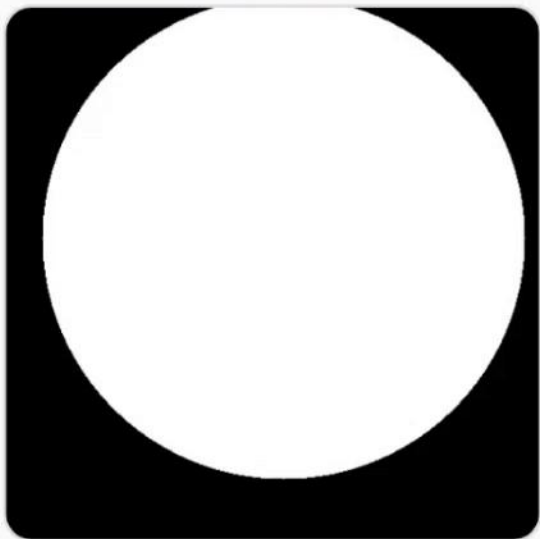


Imagen binaria donde los píxeles blancos (true) definen el área que se conservará.

3. Resultado Enmascarado



Resultado tras copiar únicamente los píxeles correspondientes a la máscara.

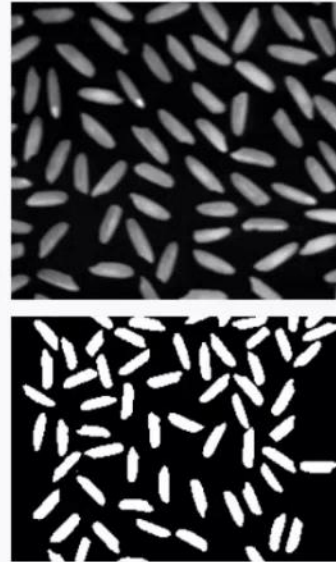
Uso: Segmentación binaria

Separación de Información

La segmentación binaria es una de las operaciones más fundamentales y utilizadas en visión artificial.

Su objetivo principal es aislar los elementos de interés (asignándoles el valor **true**) y descartar el fondo de la imagen (asignándole el valor **false**).

Caso: Granos de Arroz



Caso: Siluetas

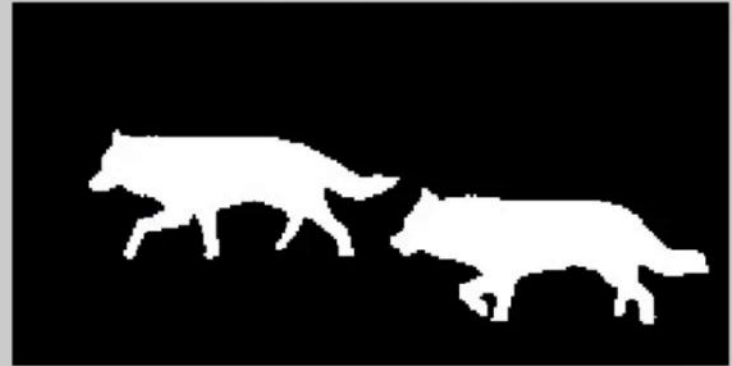


Permite extraer con alta precisión la silueta o contorno exacto de objetos para tareas de clasificación o análisis de forma.

Grayscale



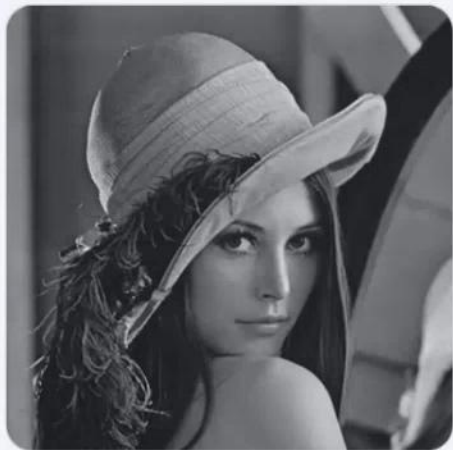
Segmented image in Binary



Segmentación binaria con umbrales

Threshold

Efecto de Umbral Creciente en Binarización



Original (Gris)



Umbral Bajo



Umbral Medio



Umbral Alto

La binarización convierte una imagen en escala de grises a blanco y negro. Al aplicar un nivel de umbral (threshold) progresivamente mayor, se requiere un nivel de brillo más alto para que cada píxel sea considerado blanco, dando como resultado imágenes con mayor predominancia de zonas oscuras.

Binarización sobre Imágenes a Color



Original (Color)



Umbral sobre Color

Umbral en Color: Un Error Común

Aplicar la función de binarización `threshold()` directamente sobre una imagen a color **no genera una imagen binaria** (blanco y negro).

En su lugar, el algoritmo opera de manera independiente en cada uno de los canales cromáticos (R, G, B), lo que produce colores artificiales inesperados. Este resultado casi siempre es consecuencia de un error común: **olvidar convertir la imagen a escala de grises** antes de binarizar.

✓ Solución correcta:

Siempre se debe convertir la imagen a escala de grises (monocromática) mediante `cvtColor()` antes de binarizar.

Efecto de Umbral Creciente en Binarización



Original (Gris)



Umbral Bajo



Umbral Medio



Umbral Alto

La binarización convierte una imagen en escala de grises a blanco y negro. Al aplicar un nivel de umbral (threshold) progresivamente mayor, se requiere un nivel de brillo más alto para que cada píxel sea considerado blanco, dando como resultado imágenes con mayor predominancia de zonas oscuras.

Binarización sobre Imágenes a Color



Original (Color)



Umbral sobre Color

Umbral en Color: Un Error Común

Aplicar la función de binarización `threshold()` directamente sobre una imagen a color **no genera una imagen binaria** (blanco y negro).

En su lugar, el algoritmo opera de manera independiente en cada uno de los canales cromáticos (R, G, B), lo que produce colores artificiales inesperados. Este resultado casi siempre es consecuencia de un error común: **olvidar convertir la imagen a escala de grises** antes de binarizar.

✓ Solución correcta:

Siempre se debe convertir la imagen a escala de grises (monocromática) mediante `cvtColor()` antes de binarizar.

inRange()

Segmentación por Color en Imágenes RGB/HSV

A diferencia de la umbralización monocromática, `inRange()` permite binarizar una imagen a color filtrando píxeles que se encuentran dentro de un rango específico de tres canales.

```
inRange(imagen_bgr, limite_inferior, limite_superior, mascara_binaria);
```



GPU-ACCELERATED LOCAL TONE-MAPPING FOR HIGH DYNAMIC RANGE IMAGES
Qiyuan Tian¹, Jiang Duan^{2*}, Guoping Qiu^{1,2*}
¹Department of Electrical Engineering, Stanford University
²Southeastern University of Finance and Economics.

GPU-ACCELERATED LOCAL TONE-MAPPING FOR HIGH DYNAMIC RANGE IMAGES
Qiyuan Tian¹, Jiang Duan^{2*}, Guoping Qiu^{1,2*}
¹Department of Electrical Engineering, Stanford University
²Southeastern University of Finance and Economics.

Resaltado
de texto

Resaltado de texto

GPU-ACCELERATED LOCAL TONE-MAPPING FOR HIGH DYNAMIC RANGE IMAGES

Qiyuan Tian¹, Jiang Du²*, Guoping Qiu³

¹Department of Electrical Engineering, Stanford University
²School of Economic Information Engineering, Southwestern University of Finance and Economics, Chengdu, China
³School of Computer Science, The University of Nottingham, UK

ABSTRACT

This paper presents a very fast local tone mapping method for displaying high dynamic range (HDR) images. Though local tone mapping operators produce better local contrast and details, they are usually slow. We have solved this problem by designing a highly parallel algorithm, which can be easily implemented on a Graphics Processing Unit (GPU) to boost high computational efficiency. At the same time, proposed method mimics the local adaptation mechanism of human visual system and thus gives good results for a variety of images.

Key Terms—Local tone mapping, high dynamic range, parallel computation, GPU, CUDA

1. INTRODUCTION

The range of a scene or an image is defined as the ratio of the highest to the lowest luminance. The real world has a very wide range of luminance (Fig. 1), spanning 10 orders of magnitude. To reproduce such a wide range of luminance on conventional digital display devices, which suffer a limited dynamic range, is a challenge. Radiance maps [1, 2], which store a sequence of low dynamic range images of the same scene taken under different exposure conditions (Fig. 1), are able to record the full range of luminance in 32-bit floating-point number. Reproduction devices such as CRT usually only 8-bit per color channel. Reproduction is the process to map the range of the radiance maps to fit into the range of the display devices, while preserving as much of the original scene as possible.

This issue by presenting a novel tone mapping method for displaying HDR images. The view of the paper is as follows. In view of previous works of tone mapping, we describe our GPU implementation in detail. Finally, we present experimental results.



Fig. 1. Selected multi-exposed image set of the same scene.

2. REVIEW OF TONE MAPPING METHODS

Tone mapping operators are usually classified as either global or local. Global tone mapping techniques apply the same appropriately designed mapping function to every pixel across the image [3] and [4] are pioneering works. The operators attempt to match the display brightness with real world sensations, and match the perceived contrast between the displayed image and the scene respectively. Later, [5] proposes a technique based on a comprehensive visual model, successfully simulating important visual effects like adaptation and color appearance. Further, [6] presents a method based on logarithmic compression of luminance values, imitating the human response to light. Recently, [7] formulates the tone mapping problem as a quantization process and employs an adaptive conscience learning strategy to obtain mapped images. Perhaps the most comprehensive technique is still that of [8], which first improves histogram equalization and then extends this idea to incorporate models of human contrast sensitivity, glare, spatial acuity, and color sensitivity effects.

Local tone mapping techniques use spatially varying mapping functions. [9-12] are based on the same principle of decomposing an image into layers and differently compressing them. Usually, layers with large features are strongly compressed to reduce the dynamic range while layers of details are untouched or even enhanced to preserve details. [13] presents a method based on a multiscale version of the Retinex theory of color vision. [14] attempts to incorporate traditional photographic techniques to the digital domain for reproducing HDR images. [15] compresses the dynamic range through the manipulation of the gradient domain in the logarithmic space. More recently, [16] proposes a novel method by adjusting the local histogram. There has been little published research to explore rendering HDR images on the GPU [17,18] and these works

GPU-ACCELERATED LOCAL TONE-MAPPING FOR HIGH DYNAMIC RANGE IMAGES

Qiyuan Tian¹, Jiang Du²*, Guoping Qiu³

¹Department of Electrical Engineering, Stanford University
²School of Economic Information Engineering, Southwestern University of Finance and Economics, Chengdu, China
³School of Computer Science, The University of Nottingham, UK

ABSTRACT

This paper presents a very fast local tone mapping method for displaying high dynamic range (HDR) images. Though local tone mapping operators produce better local contrast and details, they are usually slow. We have solved this problem by designing a highly parallel algorithm, which can be easily implemented on a Graphics Processing Unit (GPU) to boost high computational efficiency. At the same time, proposed method mimics the local adaptation mechanism of human visual system and thus gives good results for a variety of images.

Key Terms—Local tone mapping, high dynamic range, parallel computation, GPU, CUDA

1. INTRODUCTION

The range of a scene or an image is defined as the ratio of the highest to the lowest luminance. The real world has a very wide range of luminance (Fig. 1), spanning 10 orders of magnitude. To reproduce such a wide range of luminance on conventional digital display devices, which suffer a limited dynamic range, is a challenge. Radiance maps [1, 2], which store a sequence of low dynamic range images of the same scene taken under different exposure conditions (Fig. 1), are able to record the full range of luminance in 32-bit floating-point number. Reproduction devices such as CRT usually only 8-bit per color channel. Reproduction is the process to map the range of the radiance maps to fit into the range of the display devices, while preserving as much of the original scene as possible.

This issue by presenting a novel tone mapping method for displaying HDR images. The view of the paper is as follows. In view of previous works of tone mapping, we describe our GPU implementation in detail. Finally, we present experimental results.



Fig. 1. Selected multi-exposed image set of the same scene.

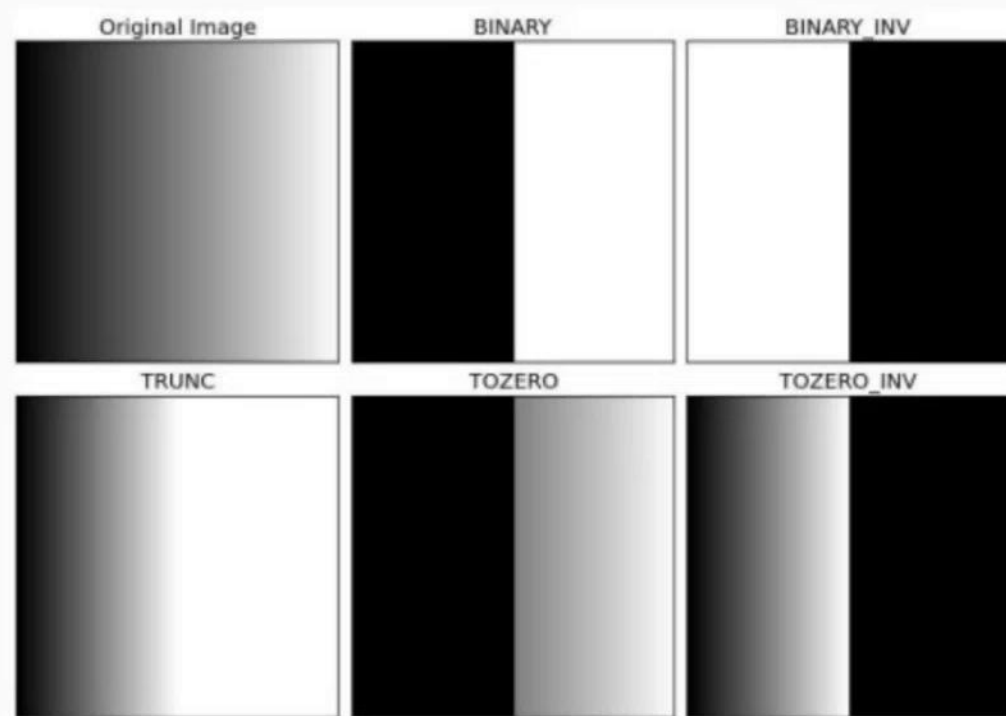
2. REVIEW OF TONE MAPPING METHODS

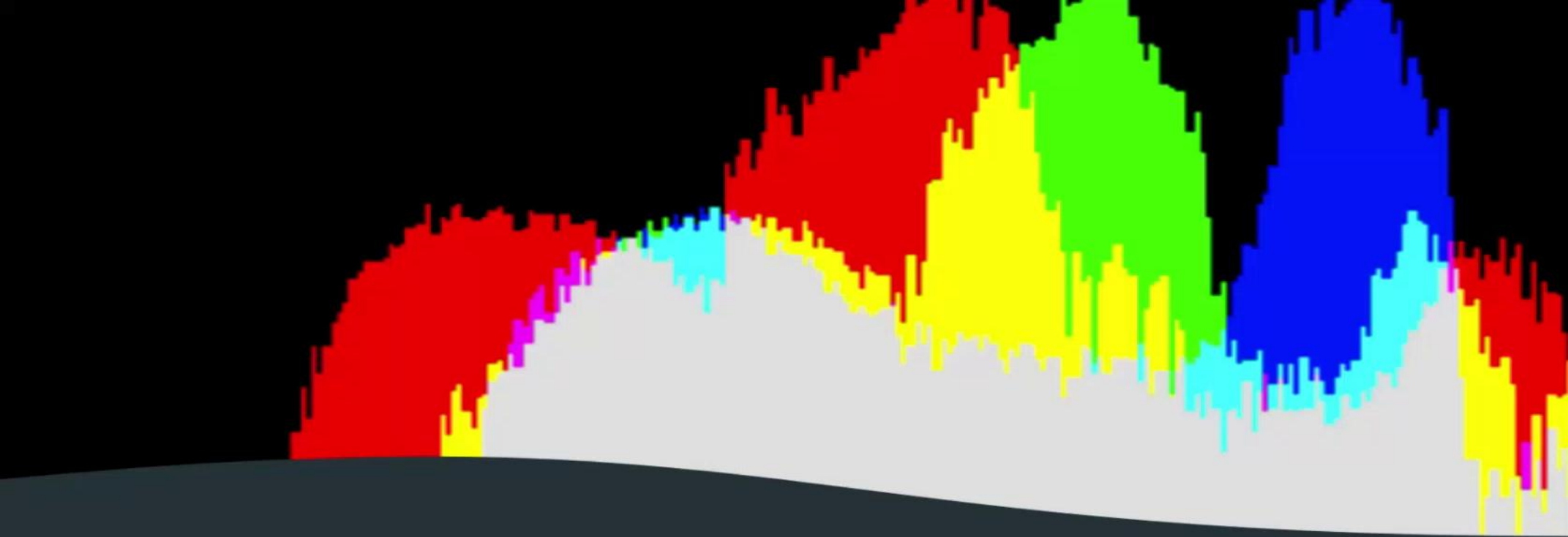
Tone mapping operators are usually classified as either global or local. Global tone mapping techniques apply the same appropriately designed mapping function to every pixel across the image [3] and [4] are pioneering works. The operators attempt to match the display brightness with real world sensations, and match the perceived contrast between the displayed image and the scene respectively. Later, [5] proposes a technique based on a comprehensive visual model, successfully simulating important visual effects like adaptation and color appearance. Further, [6] presents a method based on logarithmic compression of luminance values, imitating the human response to light. Recently, [7] formulates the tone mapping problem as a quantization process and employs an adaptive conscience learning strategy to obtain mapped images. Perhaps the most comprehensive technique is still that of [8], which first improves histogram equalization and then extends this idea to incorporate models of human contrast sensitivity, glare, spatial acuity, and color sensitivity effects.

Local tone mapping techniques use spatially varying mapping functions. [9-12] are based on the same principle of decomposing an image into layers and differently compressing them. Usually, layers with large features are strongly compressed to reduce the dynamic range while layers of details are untouched or even enhanced to preserve details. [13] presents a method based on a multiscale version of the Retinex theory of color vision. [14] attempts to incorporate traditional photographic techniques to the digital domain for reproducing HDR images. [15] compresses the dynamic range through the manipulation of the gradient domain in the logarithmic space. More recently, [16] proposes a novel method by adjusting the local histogram. There has been little published research to explore rendering HDR images on the GPU [17,18] and these works

Tipos de operación

- El comando **threshold()** tiene varios tipos de operación
- En visión artificial interesa
 - **THRESH_BINARY**
 - **THRESH_BINARY_INV** (que invierte los valores resultantes)
- Otros tipos de operación tienen aplicación en procesamiento de imágenes

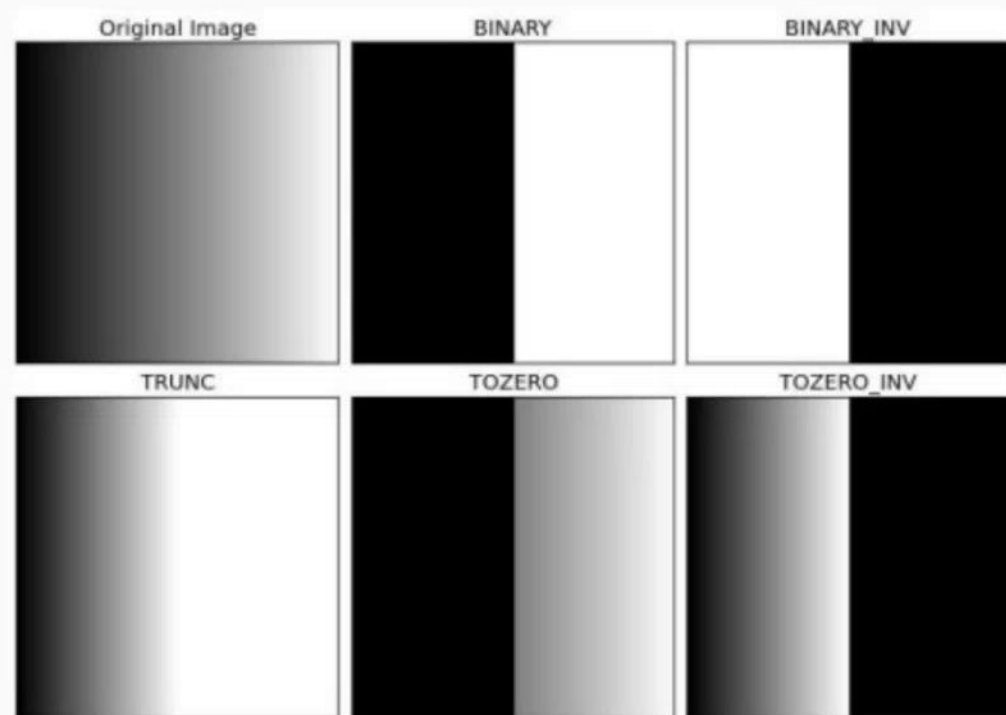


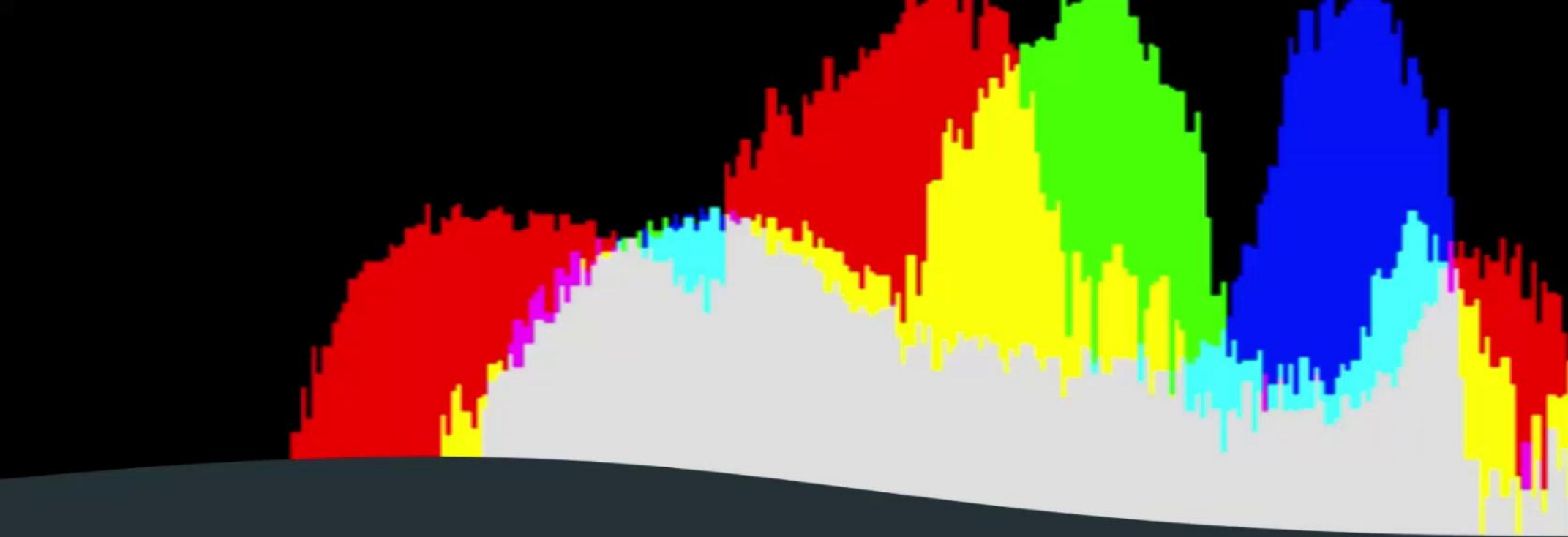


Umbrales automáticos e histogramas

Tipos de operación

- El comando **threshold()** tiene varios tipos de operación
- En visión artificial interesa
 - **THRESH_BINARY**
 - **THRESH_BINARY_INV** (que invierte los valores resultantes)
- Otros tipos de operación tienen aplicación en procesamiento de imágenes





Umbrales automáticos e histogramas

Algunos embeddings

Rostros

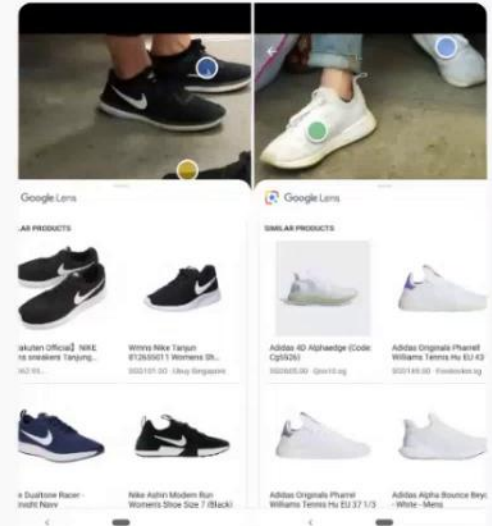
FaceNet y ArcFace describen rostros para su reconocimiento de manera precisa.

Clasificadores

Los clasificadores basados en deep learning usan sus propios embeddings internos.

Google Lens

Busca productos similares comparando embeddings visuales de las imágenes.



Histograma de intensidades

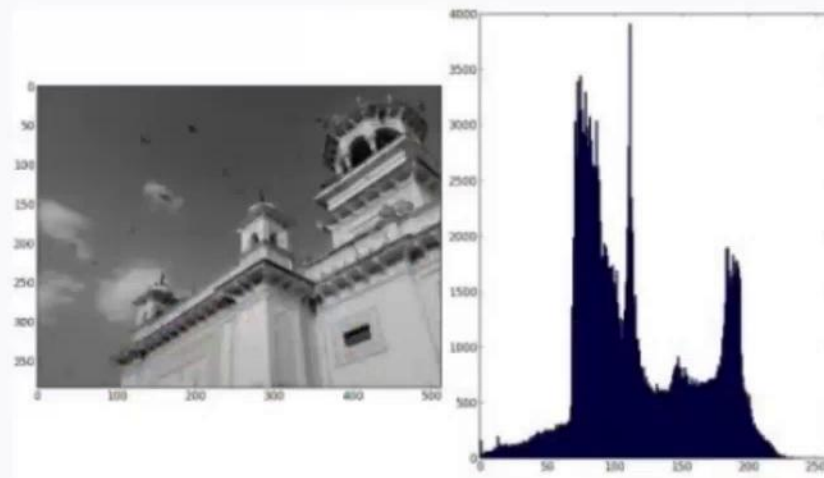
RANGO DE VALORES

Las intensidades de los píxeles varían en un rango estándar entre **0 y 255**.

HISTOGRAMA

Cantidad de píxeles para cada valor de 0 a 255

PROCESAMIENTO Y DISTRIBUCIÓN



Mapeo de Intensidades

La gráfica ilustra cómo se distribuyen los tonos de la imagen, desde el negro absoluto (0) hasta el blanco puro (255).

Detección automática de umbral

Análisis Bimodal

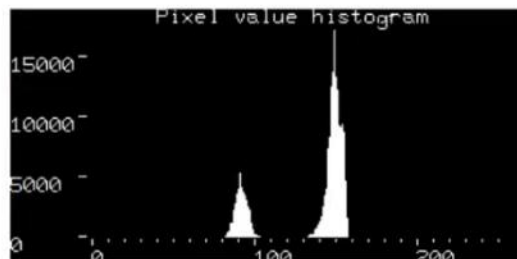
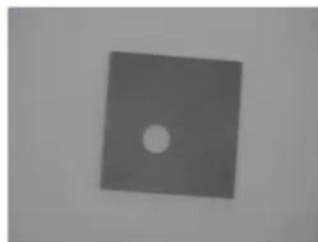
01 / SEGMENTACIÓN

Se calcula un valor de umbral para separar con precisión el objeto de interés del fondo de la escena.

02 / BÚSQUEDA DEL VALLE

Con una distribución bimodal, el umbral óptimo se localiza en el punto mínimo entre ambas modas de intensidad.

PROCESAMIENTO Y HISTOGRAMA



Mínimo del valle

El punto más bajo entre los dos picos del histograma define el límite ideal para la binarización.

Histograma de intensidades

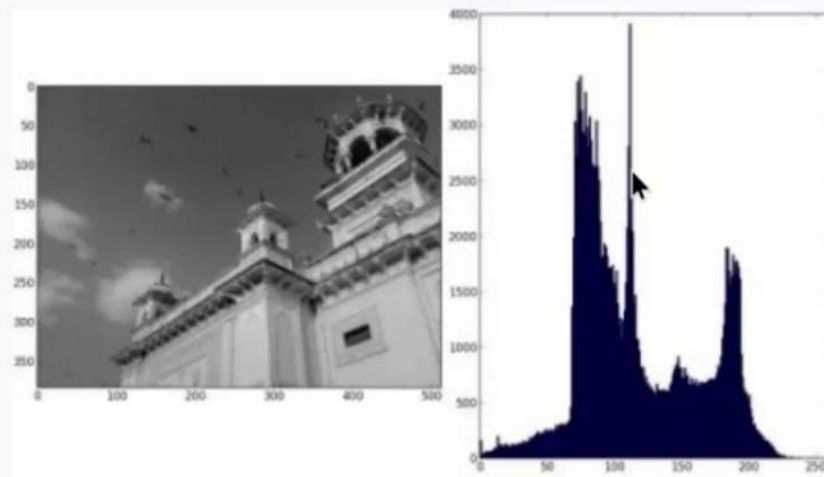
RANGO DE VALORES

Las intensidades de los píxeles varían en un rango estándar entre **0 y 255**.

HISTOGRAMA

Cantidad de píxeles para cada valor de 0 a 255

PROCESAMIENTO Y DISTRIBUCIÓN



Mapeo de Intensidades

La gráfica ilustra cómo se distribuyen los tonos de la imagen, desde el negro absoluto (0) hasta el blanco puro (255).

Detección automática de umbral

Análisis Bimodal

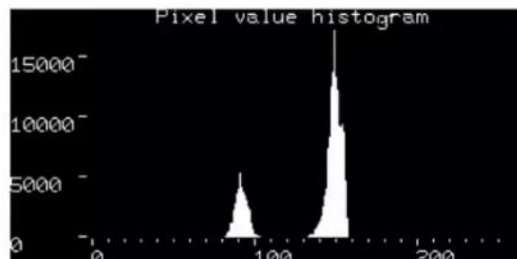
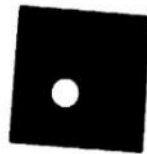
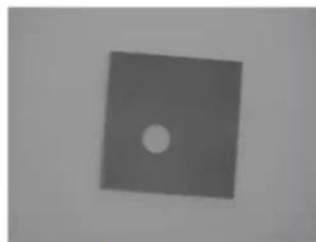
01 / SEGMENTACIÓN

Se calcula un valor de umbral para separar con precisión el objeto de interés del fondo de la escena.

02 / BÚSQUEDA DEL VALLE

Con una distribución bimodal, el umbral óptimo se localiza en el punto mínimo entre ambas modas de intensidad.

PROCESAMIENTO Y HISTOGRAMA



Mínimo del valle

El punto más bajo entre los dos picos del histograma define el límite ideal para la binarización.

Umbralización automática

Combinación de banderas (flags)

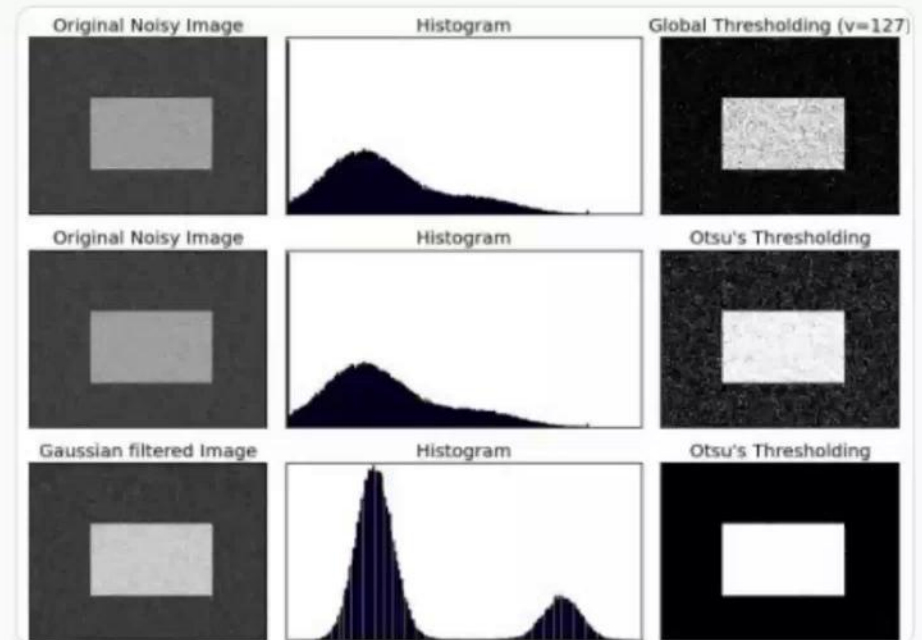
El argumento "**tipo de operación**" se puede combinar con alguno de estos métodos de umbralización automática:

THRESH_OTSU

THRESH_TRIANGLE

Cálculo Óptimo Autónomo

Estos algoritmos analizan la imagen internamente para calcular el umbral óptimo, ignorando cualquier valor manual proporcionado.



Umbral adaptativo

- **adaptiveThreshold()**
- Un umbral constante no es efectivo cuando la iluminación varía en la imagen
- Los algoritmos de umbral adaptativo calculan el umbral sobre una ventana deslizante
- No requiere un argumento "umbral", pero sí un "tamaño de bloque" de la ventana deslizante
- Es notablemente más lento que **threshold()**

Original Image



Global Thresholding ($\nu = 127$)



Adaptive Mean Thresholding



Adaptive Gaussian Thresholding





Blobs



Guardado en Drive

Archivo Editar Ver Insertar Formato Diapositiva Estructura Herrami



Presentación de diapositivas



Actualizar



Ajustar



Tr



Fondo

Diseño

Tema

Transición



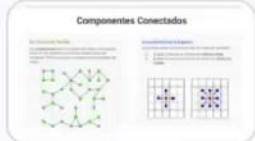
1



2



3



4



Haz clic para agregar notas del orador



Diapositiva



Contenido multimedia



Cargas



Grabar



Transformar



Blob



Visión Artificial
Alejandro Silvestri

Paso 1

Primera etapa de interpretación

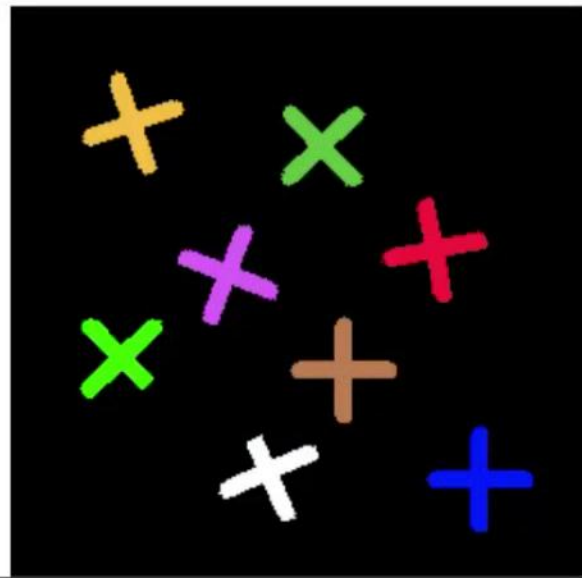
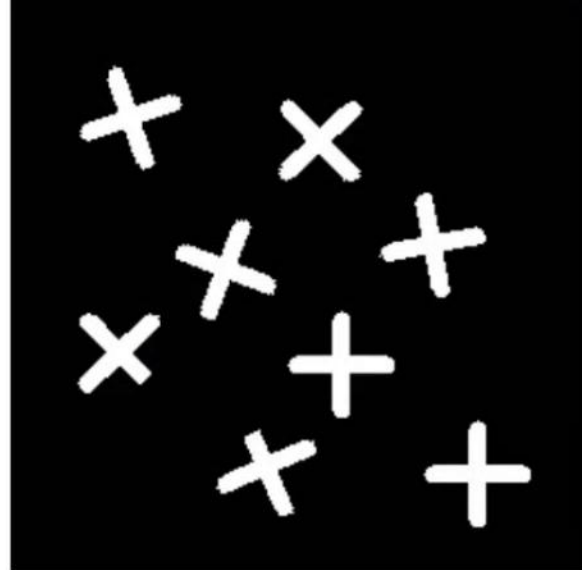
Visión Artificial

Es el primer paso genuino, donde se extraen figuras individuales a partir de los píxeles, logrando un primer nivel de interpretación.

Detección de Blobs

Luego de aplicar `connectedComponents()`, la máquina ve figuras individuales (blobs) en lugar de píxeles sueltos:

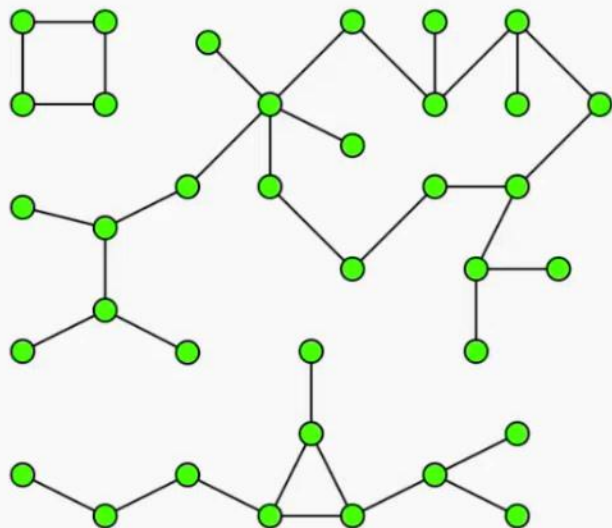
- **Individualiza:** Identifica cada elemento independiente.
- **Parametriza:** Mide y caracteriza sus propiedades.



Componentes Conectados

En Teoría de Grafos

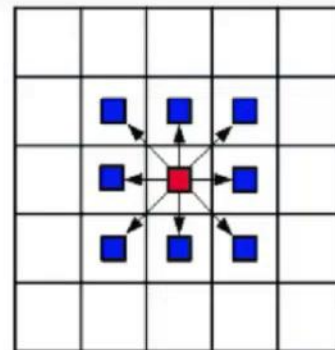
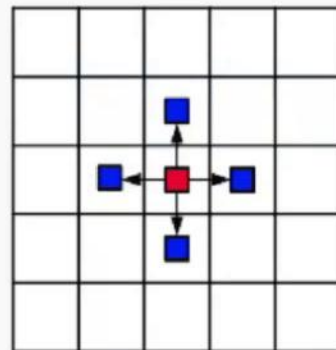
Un **componente** es un conjunto de nodos conectados entre sí, sin conexión con otros nodos fuera del conjunto. Forma un grupo completamente aislado del resto.



Conectividad en Imágenes

Los píxeles actúan como nodos con dos tipos de vecindad:

- **4-vías:** Conectados si tienen un **lado en común**.
- **8-vías:** Conectados si tienen al menos un **vértice en común**.



Grupos y etiquetado

El Algoritmo y Proceso

connectedComponents() produce una imagen segmentada, donde cada píxel recibe una etiqueta numérica.

Las etiquetas son números enteros correlativos, comenzando en **0 para el fondo**.

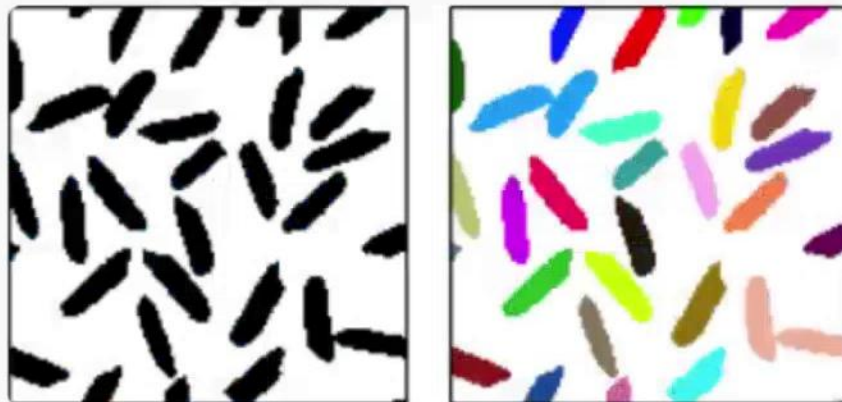
El etiquetado consiste en:

- Asignar un **identificador o etiqueta** diferente a cada grupo.
- Asignar luego dicho identificador a cada uno de los píxeles conectados.

Visualización de Resultados

Los colores indican que la computadora ha logrado **individualizar las figuras** con precisión.

Nota: Los colores se visualizan de manera intuitiva utilizando la función **colorMap()**.



Contornos y Análisis de Blobs

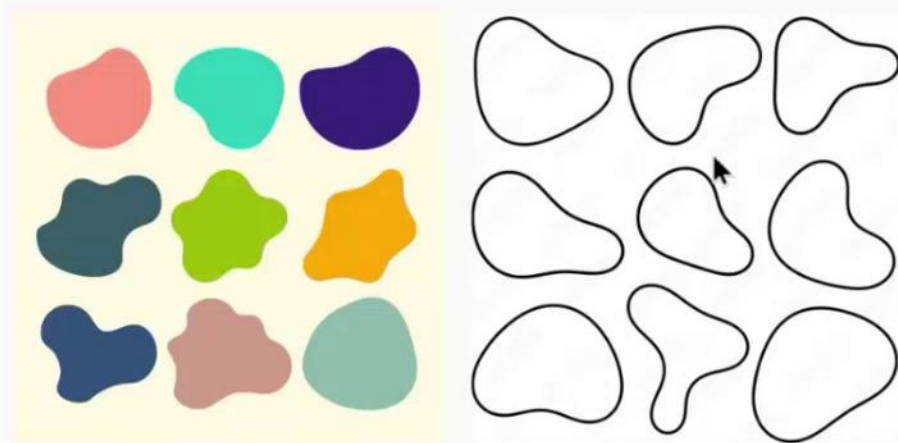
Definición de Contornos

Un contorno es el primer paso crítico para extraer información útil de un blob segmentado:

- **Definición matemática:** Un blob se puede definir y mapear completamente por su contorno exterior.
- **No es una imagen:** Representa un conjunto de coordenadas exactas de los píxeles que lo forman.
- **Paso a información:** Transforma una matriz de píxeles en datos geométricos analizables.

De la Imagen a los Blobs

- **Áreas Amorfas:** Los blobs son componentes conectados continuos sin forma predefinida.
- **Transición:** Las imágenes segmentadas siguen siendo imágenes; requieren contornos para ser datos.





Contornos



Guardado en Drive

Archivo Editar Ver Insertar Formato Diapositiva Estructura Herrami



Presentación de diapositivas



Actualizar



Ajustar



Tr



Fondo

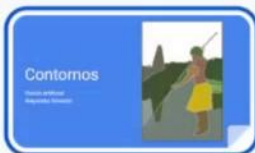
Diseño

Tema

Transición



1



2



3



4



Diapositiva



Contenido multimedia



Cargas



Grabar



Transformar



Haz clic para agregar notas del orador



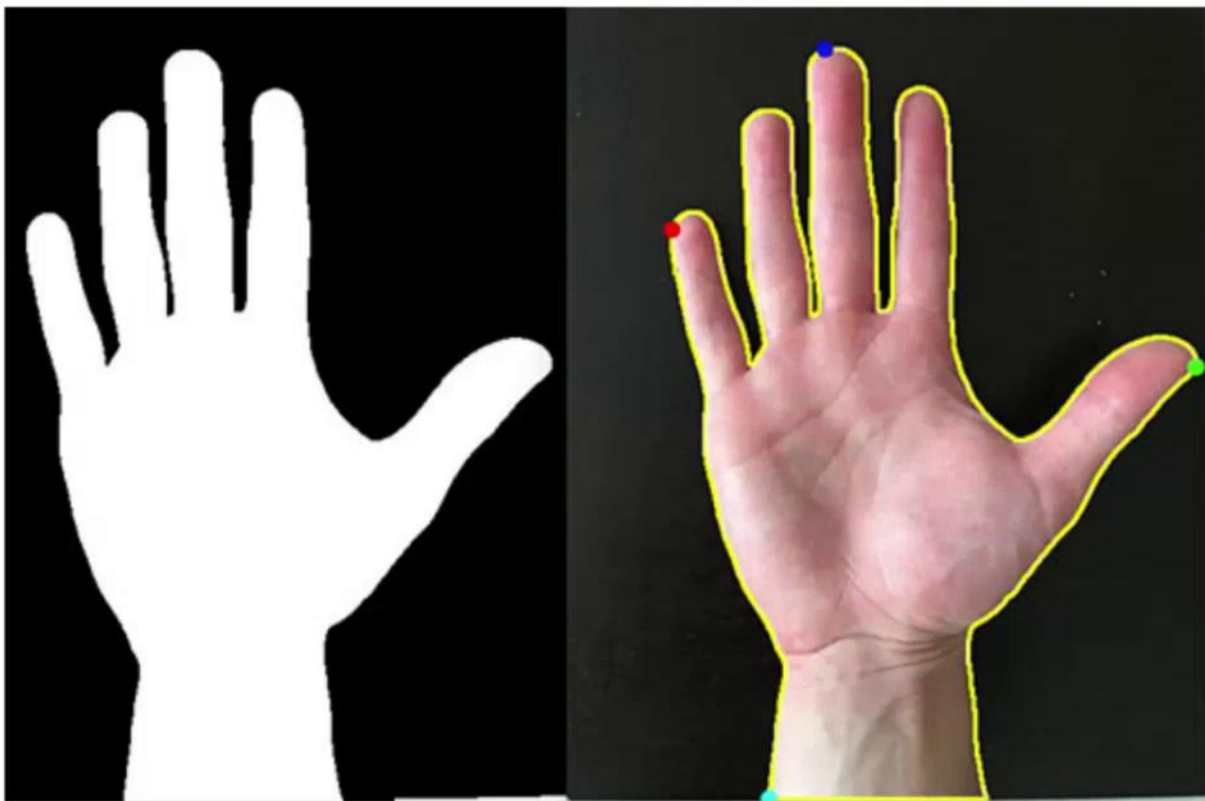
Contornos

Visión artificial
Alejandro Silvestri



Contornos ¿para qué?

- Las imágenes binarias contienen figuras, formas
 - Componentes conectados permiten etiquetarlas
- Otra manera de expresar las mismas formas es a través de sus contornos
 - Los contornos son líneas cerradas que delimitan el borde de las figuras

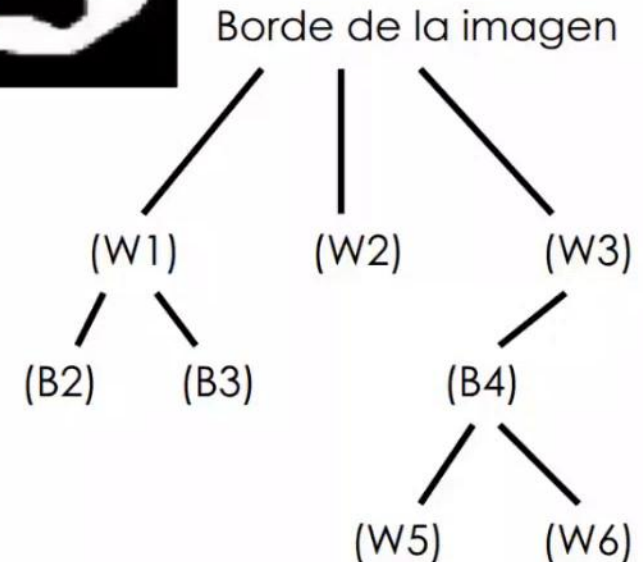
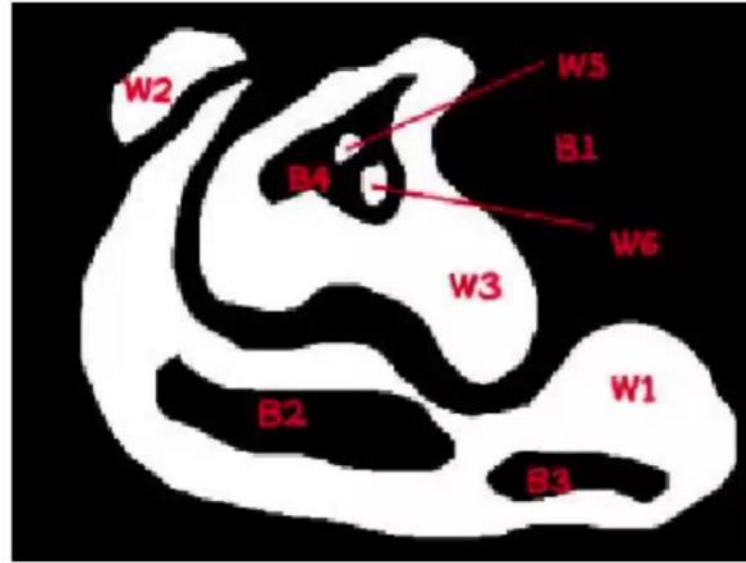


Contornos ¿por qué?

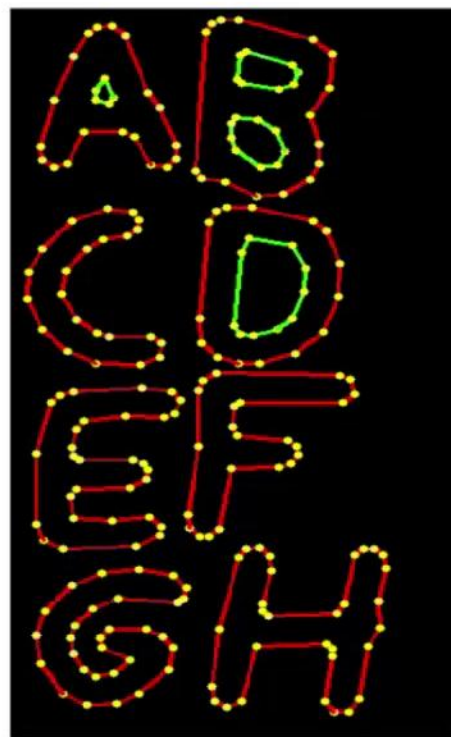
- Más compactos que las imágenes
- Naturalmente etiquetados
 - equivale al análisis de componentes conectados
- Jerárquicos
 - la jerarquía indica si hay contornos anidados
- Permiten análisis de formas
 - más eficientes que las imágenes binarias
- Fáciles de anotar sobre una imagen

Contornos básicos

- Son una secuencia de puntos (coordenadas) que forman una línea cerrada que define una figura
- Figuras complejas tienen contornos anidados, que se representan por medio de una jerarquía



Ejemplo



Obtener
contornos
de una imagen



findContours()

- **Requiere**
 - **Imagen binaria**
 - o etiquetada por componentes conectados
 - **Modo**
 - jerarquía adoptada
 - **Método** de aproximación
- **Produce**
 - **Contornos**
 - un array de contornos
 - cada contorno es a su vez un array de puntos
 - **Jerarquía**
 - una construcción de arrays conteniendo índices a los contornos, cuya interpretación depende del modo elegido.

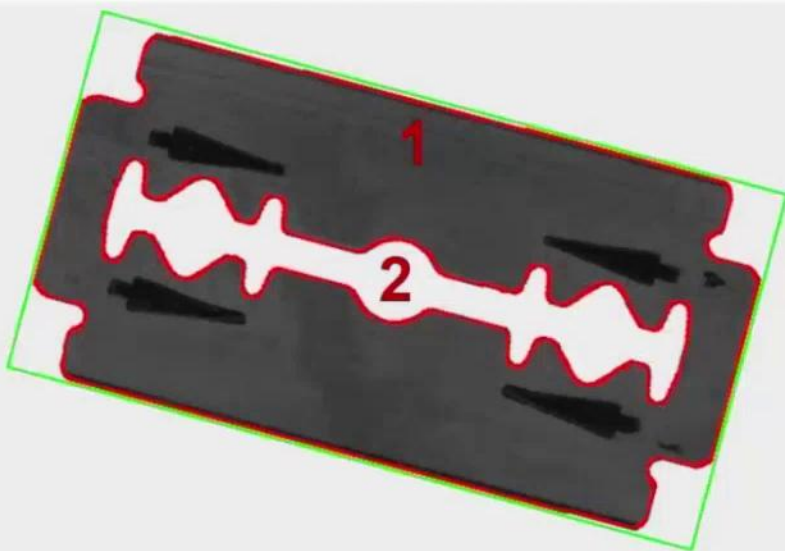
Metrología



Valores fácilmente medibles

Area 1: 522314
Perim. 1: 3265
Width. 1: 882
Height 1: 752

Area 2: 21115
Perim. 2: 1386
Width. 2: 453
Height 2: 81



- `arcLength()`
 - **perímetro** de un contorno, en píxeles
- `contourArea()`
 - **superficie** que encierra un contorno, en píxeles
 - muchas veces el ruido produce contornos de pequeña superficie, que se descartan fácilmente con este valor

`minAreaRect()`

- rectángulo rotado mínimo
- útil para obtener **largo** y **ancho**

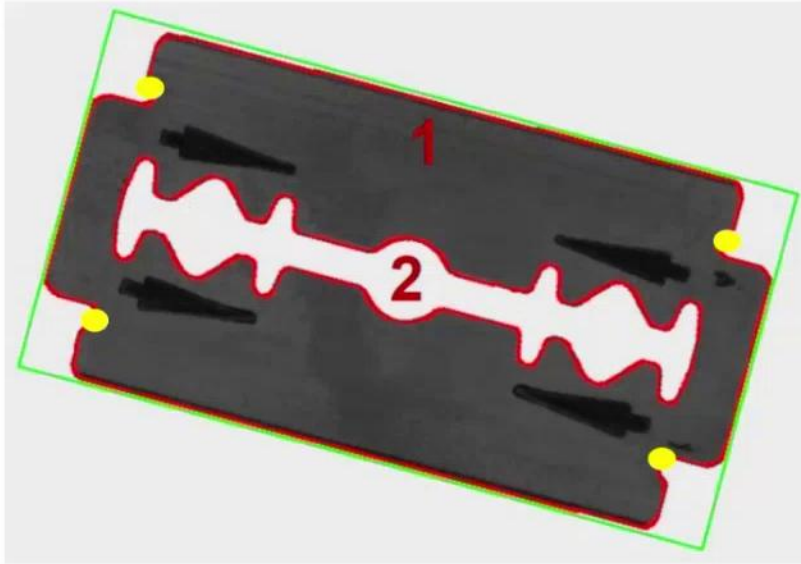
Convexidades

`convexityDefects()`

- **cantidad de convexidades** de un contorno (cantidad de picos)



Comparación con plantillas



- Para detectar defectos se comparan un contorno relevado contra un **contorno de referencia**
 - Alineados ambos contornos, se mide la distancia de **ciertos puntos elegidos** de la plantilla, al contorno relevado
- **pointPolygonTest(punto)**
 - **distancia** del punto dado al contorno
 - el signo indica si el punto está **dentro o fuera** del contorno
 - en metrología se usa para medir la disparidad entre el contorno de una pieza y puntos por donde debería pasar según especificaciones

Comparación con plantillas:

matchShapes

CONTOURS_MATCH_I1

Python: cv.CONTOURS_MATCH_I1

$$I_1(A, B) = \sum_{i=1 \dots 7} \left| \frac{1}{m_i^A} - \frac{1}{m_i^B} \right|$$

CONTOURS_MATCH_I2

Python: cv.CONTOURS_MATCH_I2

$$I_2(A, B) = \sum_{i=1 \dots 7} |m_i^A - m_i^B|$$

CONTOURS_MATCH_I3

Python: cv.CONTOURS_MATCH_I3

$$I_3(A, B) = \max_{i=1 \dots 7} \frac{|m_i^A - m_i^B|}{|m_i^A|}$$

- compara dos contornos
- el resultado se denomina **distancia**
 - la distancia expresa la disparidad entre los contornos
 - cuanto más diferentes son los contornos, mayor es la distancia
 - un contorno comparado con sí mismo resulta en una **distancia cero**
- la comparación no se realiza punto a punto, sino de manera indirecta empleando **momentos invariantes de Hu**
 - son invariantes a la rotación, traslación y escala
- se utiliza en clasificadores, para reconocer formas
 - se compara el contorno contra varias plantillas
 - se consideran las distancias menores a un umbral
 - se elige la plantilla de la menor distancia, siempre que sea menor a al umbral dado

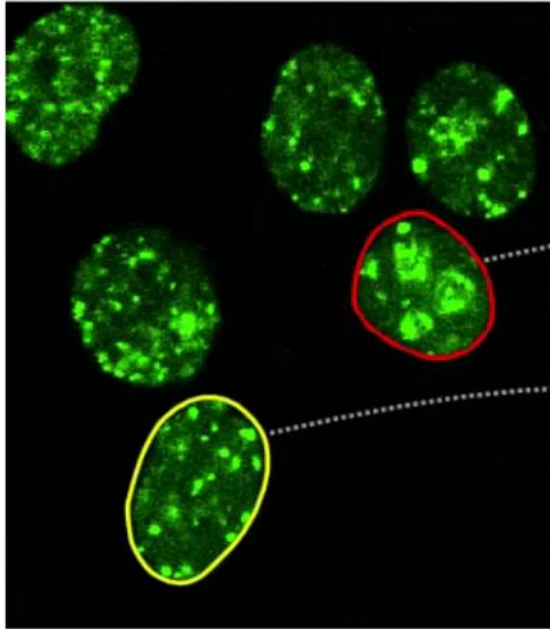
Anotaciones



drawContours()

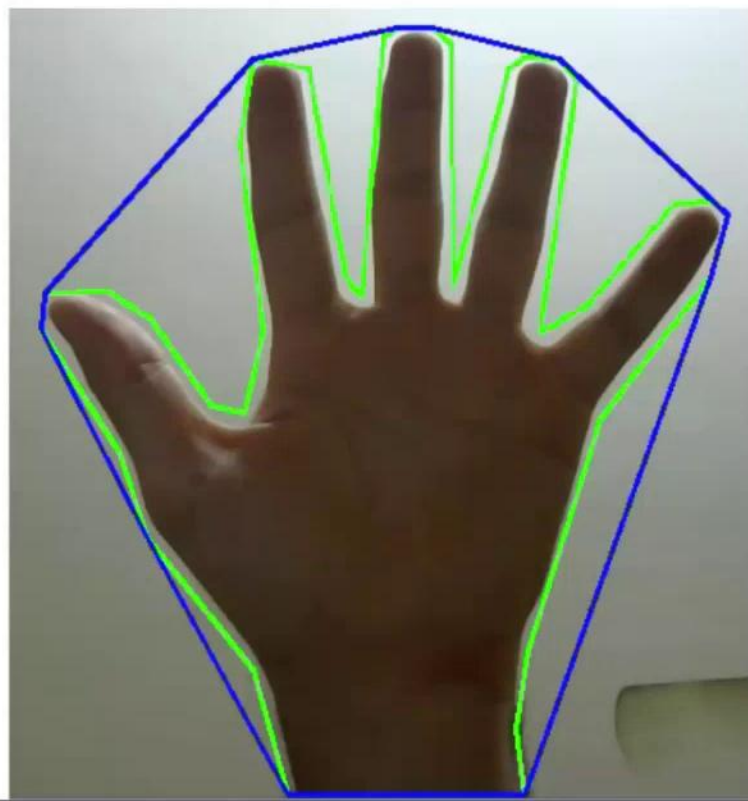
- Dibuja el contorno sobre una imagen
- Requiere
 - la **imagen** sobre la que dibujar
 - array de **contornos**
 - **índice** del contorno a dibujar, o negativo para dibujar todos los contornos
 - **decoración**: color y tipo de línea

drawContours()



- Dibuja el contorno sobre una imagen
- Requiere
 - la **imagen** sobre la que dibujar
 - array de **contornos**
 - **índice** del contorno a dibujar, o negativo para dibujar todos los contornos
 - **decoración**: color y tipo de línea

Anotaciones sobre contornos



- `boundingRect()`
 - calcula el rectángulo que contiene completamente al contorno
- `convexHull()`
 - computa la cuerda que encierra al contorno
- `minEnclosingCircle()`
- `minEnclosingTriangle()`

Enlaces

- Referencia
 - [findContours\(\)](#)
 - [approxPolyDP\(\)](#)
 - [drawContours\(\)](#)
 - [matchShapes\(\)](#)
- Tutoriales y ejemplos
 - [Varios sobre contornos, en Python](#)



Momentos



Guardado en Drive

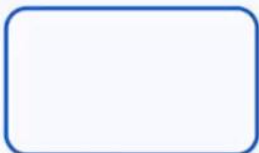
Archivo Editar Ver Insertar Formato Diapositiva Estructura Herramientas Extensiones ...



Presentación de diapositivas



1



2



3



4



Haz clic para agregar notas del orador



Diapositiva



Contenido multimedia



Cargas

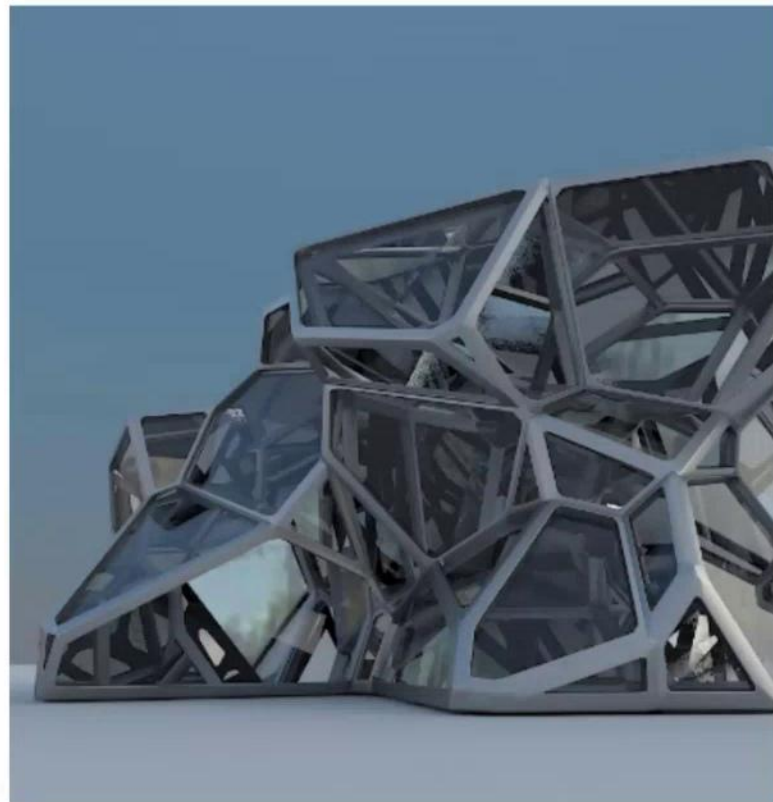


Grabar



Transformar

Reconocimiento de formas 2D



Visión artificial
Alejandro Silvestri

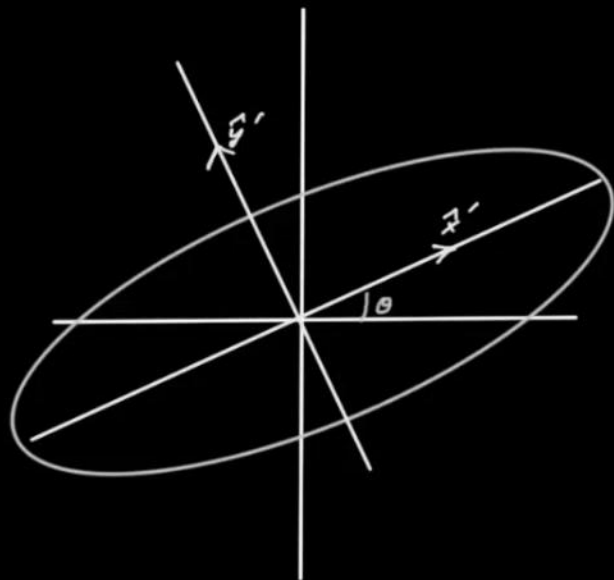
Reconocimiento = clasificación

-
- Las formas se representan por su **contorno**
 - Reconocer consiste en identificar formas **parecidas**
 - Las formas a reconocer se denominan **clases**
 - Cada clase tiene un **patrón** de referencia
 - Formas parecidas a ese patrón pertenecen a la misma clase
 - El proceso para identificar a qué clase pertenece una forma se denomina **clasificación**
 - este nombre trasciende la visión artificial, y es un problema general de machine learning

Características aka *Features*

- El término fue adoptado por la jerga de visión artificial y de machine learning
- Son valores de una forma, obtenidos a partir de su análisis
- Se los consideran **parámetros** de la forma
- Las características intuitivas corresponden a entidades geométricas
 - ej: área, perímetro
- También hay características abstractas, pero con propiedades matemáticas útiles
 - ej: invariante de Hu

Algunas características de las formas



- Centroide
 - posición
 - coordenadas x, y del baricentro
 - Orientación
 - ángulo de rotación respecto de una orientación arbitraria tomada como referencia
 - Tamaño
 - escalamiento
- Excentricidad
 - elongación u oblongamiento
 - Concavidades
 - o convexidades
 - Perímetro del contorno
 - Características abstractas

¿Qué significa “parecido”?

-
- El parecido entre dos formas es una definición ad hoc para la solución particular que se está buscando
 - La computadora detecta el parecido cuando encuentra varias **características relevantes parecidas**
 - el parecido se evalúa con una tolerancia
 - $|c1 - c2| < \text{umbral}$
 - Algunas características propios de una forma, que usualmente no se incluye en el parecido
 - posición, coordenadas en la imagen
 - rotación, orientación
 - escala, tamaño

Descriptores

- El conjunto de **características** relevantes que hacen al parecido y que permiten distinguir una clase de otra se denomina **descriptor**
 - estos aspectos relevantes se expresan numéricamente como **características**, **parámetros** o **features**
- Por lo tanto, un descriptor es un conjunto de valores, usualmente **abstractos**, elegidos para el problema particular, con los que se pueden distinguir clases
- Descriptores parecidos corresponden a la misma clase, descriptores diferentes no

Invarianza



- Se buscan características que no se vean afectadas por
 - la posición
 - la rotación
 - la escala
- Una característica cuyo valor no depende de otra se denomina invariante a esa otra característica
 - el tamaño es invariante a la posición
 - el perímetro es invariante a la rotación pero no a la escala



**Reconocimiento
de formas**

Reconocimiento de formas

-
- Las formas se reconocen por sus invariantes de Hu
 - El conjunto de invariantes de Hu es un **descriptor de formas**
 - Usos en reconocimiento
 - de **caracteres** (una vez individualizados)
 - de **patentes** vehiculares
 - de **formas rígidas** arbitrarias
 - se calculan sus invariantes
 - se comparan con los de otras imágenes
 - su variabilidad está en el orden del 1%

matchShapes()

- Compara las formas de dos contornos y devuelve una **distancia**
 - A mayor distancia, mayor es la diferencia de las formas
- Cálculo de la distancia
 - Exclusivamente en base a los invariantes de Hu
 - Cada array de invariantes de Hu forma un vector de 7 dimensiones
 - La **distancia** se computa con el método elegido, de los varios disponibles en la función

```
distancia = cv.matchShapes(contorno1, contorno2, cv.CONTOURS_MATCH_I1, 0.0)
```


Parametrización adicional

- Los invariantes de Hu son excelentes descriptores de formas
- Otros parámetros relevantes, no necesariamente invariantes, son
 - Área
 - Centroide
 - Orientación
 - Excentricidad
- Parámetros exógenos (obtenidos de contornos, no de momentos)
 - Perímetro
 - Convexidades

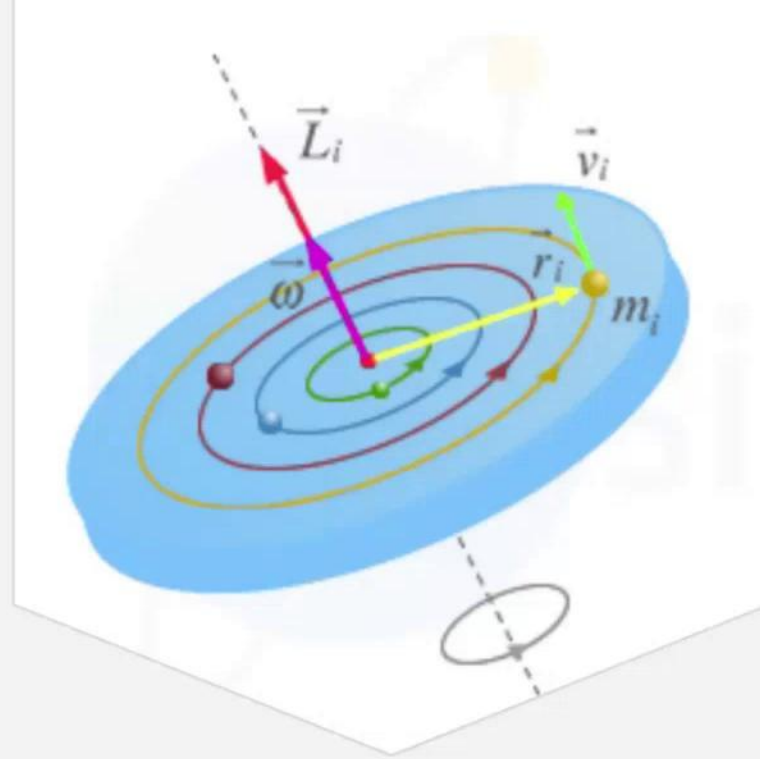
Más características de contornos

- Relación de aspecto
 - del bounding rectangle
- Extensión
- Solidez
- Color promedio
 - obtenido de la imagen original, en la región del contorno

$$\textit{Aspect Ratio} = \frac{\textit{Width}}{\textit{Height}}$$

$$\textit{Extent} = \frac{\textit{Object Area}}{\textit{Bounding Rectangle Area}}$$

$$\textit{Solidity} = \frac{\textit{Contour Area}}{\textit{Convex Hull Area}}$$



Momentos

Momentos de inercia

Momentos de inercia

- El nombre proviene de la física
- Los momentos son **evaluaciones polinómicas** realizadas sobre imágenes
 - Nos interesan especialmente los momentos sobre imágenes binarias
 - Más específicamente, momentos sobre los **componentes conectados** que encierran los contornos
 - Su **significado es más abstracto** cuando mayor es el orden de los polinomios
- Sus valores dependen de:
 - la posición
 - el tamaño
 - la rotación
 - **la forma**
- Son **características** y se usan en **descriptores** de formas

Propiedades de la estructura Moments

spatial moments

double	m00
double	m10
double	m01
double	m20
double	m11
double	m02
double	m30
double	m21
double	m12
double	m03

central moments

double	mu20
double	mu11
double	mu02
double	mu30
double	mu21
double	mu12
double	mu03

central normalized moments

double	nu20
double	nu11
double	nu02
double	nu30
double	nu21
double	nu12
double	nu03

**Momentos
especiales M**



Momentos espaciales M de una imagen

- x e y son las coordenadas de píxeles
- $I(x,y)$ es la intensidad (escala de grises)
 - En imágenes binarias se interpreta como 0 y 1
- $i + j$ es el orden del momento

$$M_{ij} = \sum_x \sum_y x^i y^j I(x, y)$$

Momento ponderado por intensidad de los píxeles

$$M_{ij} = \sum x^i y^j$$

Momento sobre imagen binaria

Momentos espaciales M de orden 0 y 1

- M_{00} en una imagen binaria es el **área**
 - Se utiliza como indicación del **tamaño** de la figura

$$M_{ij} = \sum x^i y^j$$

- $(M_{10}, M_{01})/M_{00}$ es el **centroide**
 - o **baricentro**, o centro de masa
 - se utiliza frecuentemente para representar la **posición** de la figura o contorno

$$\text{Centroid: } \{\bar{x}, \bar{y}\} = \left\{ \frac{M_{10}}{M_{00}}, \frac{M_{01}}{M_{00}} \right\}$$

**Momentos
centrales μ**



Momentos centrales μ

- Se consideran las **coordenadas relativas al centroide**
- De este modo se vuelven **invariantes a la traslación**

$$\mu_{pq} = \sum (x - \bar{x})^p (y - \bar{y})^q$$

Cómputo de autovalores y autovectores

```
#de una matriz simétrica
```

```
retval, autovalores, autovectores = cv.eigen(matriz)
```

```
# de la matriz de covarianza
```

```
retval, autovalores, autovectores = cv.eigen(  
    np.array([[mu20, mu11], [mu11, mu02]]))  
)
```

**Momentos
normales η**



Momentos normales η

- Además de ser invariantes a la posición (como los momentos centrales), también son **invariantes al tamaño**
- Se utilizan para reconocer formas que puede presentarse en diferentes tamaños, pero no rotados
 - Por ejemplo, gente caminando



$$\eta_{ij} = \frac{\mu_{ij}}{\mu_{00} \left(1 + \frac{i+j}{2}\right)}$$

Momentos normales η

- Se independizan de
 - la posición
 - el tamaño
- Momentos normales de 2º orden distinguen
 - orientación
 - excentricidad
 - de la misma manera que los momentos centrales
- Momentos normales de 3º orden
 - codifican información sobre la **forma**

A ginger and white kitten is lying on its back on a white surface. Its front paws are raised near its eyes, and its hind legs are also raised. The kitten has large, dark eyes and a pink nose. A semi-transparent white rectangular box is overlaid on the left side of the image, containing the text "Invariantes de Hu".

Invariantes de Hu

Invariantes de Hu

		Image	H[0]	H[1]	H[2]	H[3]	H[4]	H[5]	H[6]
Original	K		2.78871	6.50638	9.44249	9.84018	-19.593	-13.1205	19.6797
	S		2.67431	5.77446	9.90311	11.0016	-21.4722	-14.1102	22.0012
	S		2.67431	5.77446	9.90311	11.0016	-21.4722	-14.1102	22.0012
	S		2.65884	5.7358	9.66822	10.7427	-20.9914	-13.8694	21.3202
	S		2.66083	5.745	9.80616	10.8859	-21.2468	-13.9653	21.8214
	S		2.66083	5.745	9.80616	10.8859	-21.2468	-13.9653	-21.8214
Reflexión									

Invariantes de Hu

`hu = cv.HuMoments(Momentos)`

$$hu[0] = \eta_{20} + \eta_{02}$$

$$hu[1] = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

$$hu[2] = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$$

$$hu[3] = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$$

$$hu[4] = (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] + (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

$$hu[5] = (\eta_{20} - \eta_{02})[(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03})$$

$$hu[6] = (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] - (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

η_{ji} corresponde a `Momentos::nuji`

están calculados exclusivamente a partir de los momentos normales

Enlaces

- Referencia
 - [moments\(\)](#)
 - [Moments](#)
 - [HuMoments\(\)](#)
 - [matchShapes\(\)](#)
 - [eigen\(\)](#)
- Tutoriales y ejemplos
 - [Invariantes de Hu](#)
 - [OpenCV: Contour Properties](#)
- Documentos
 - [Momentos](#) en Wikipedia
 - [paper de invariantes de Hu](#) 1962



Operaciones morfológicas



Guardado en Drive

Archivo Editar Ver Insertar Formato Diapositiva Estructura ...



Presentación de diapositivas



Actualizar



Ajustar



Fondo

Diseño

Tema

Transición



1



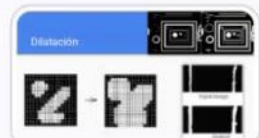
2



3

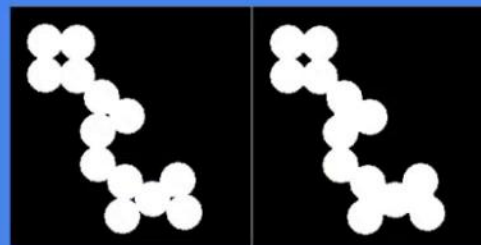


4



Operaciones morfológicas

Visión Artificial
Alejandro Silvestri



Diapositiva



Contenido multimedia



Cargas



Grabar

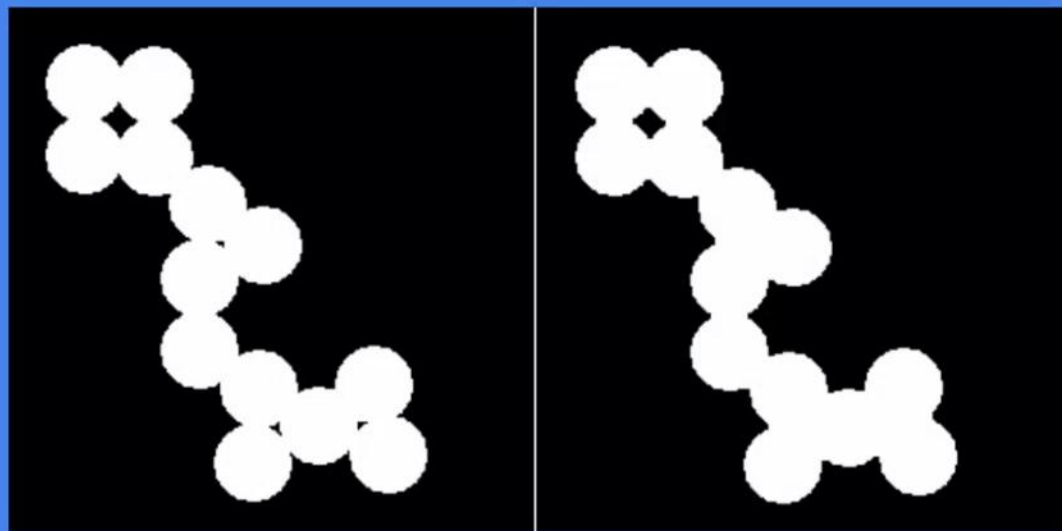


Transformar

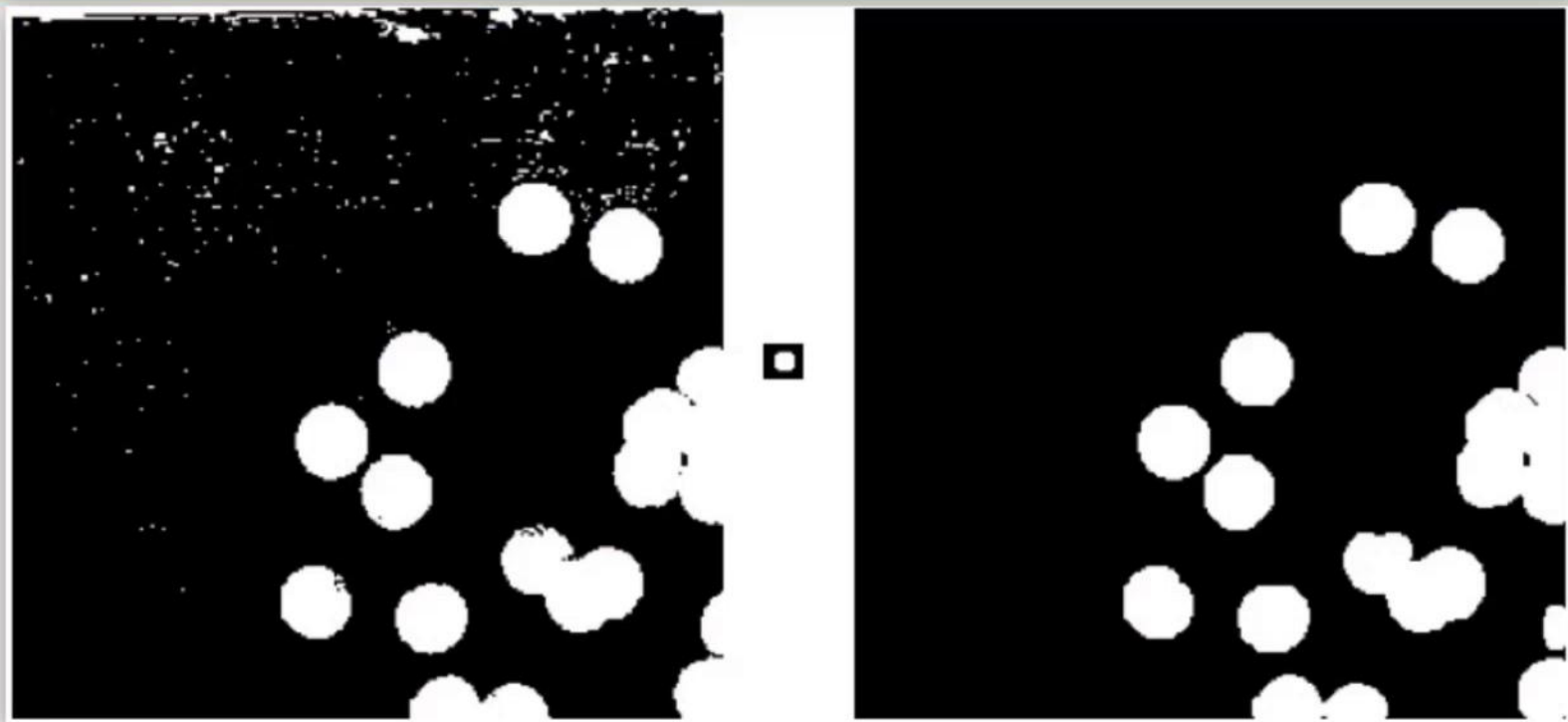
Haz clic para agregar notas del orador

Operaciones morfológicas

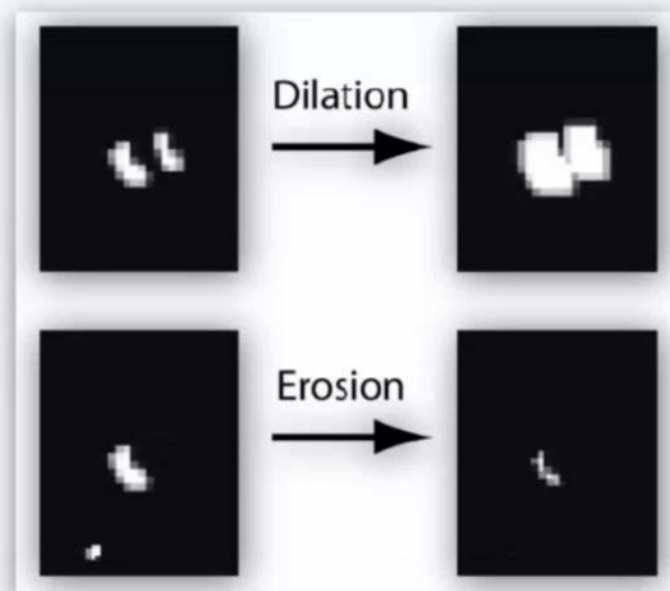
Visión Artificial
Alejandro Silvestri



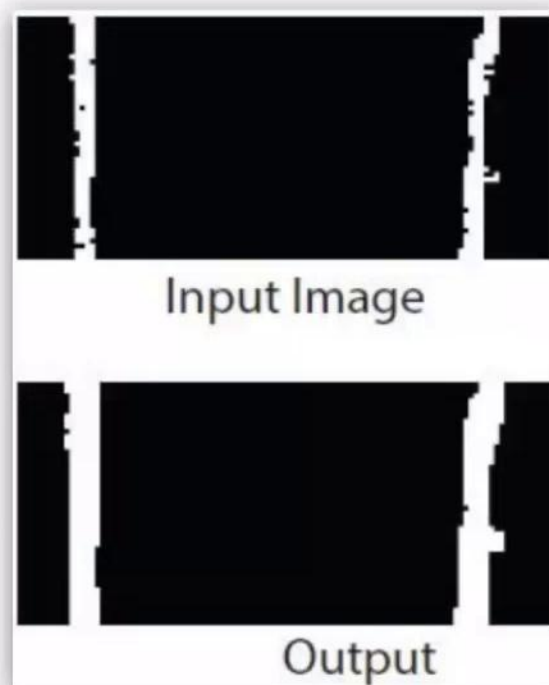
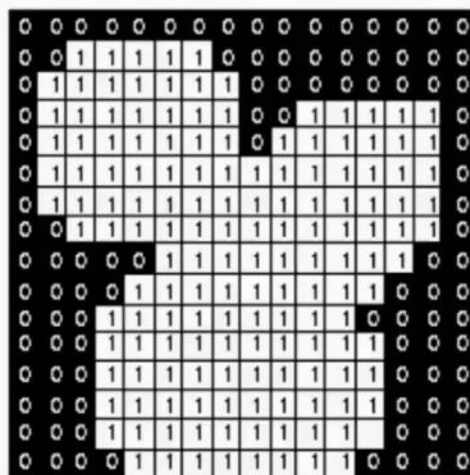
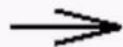
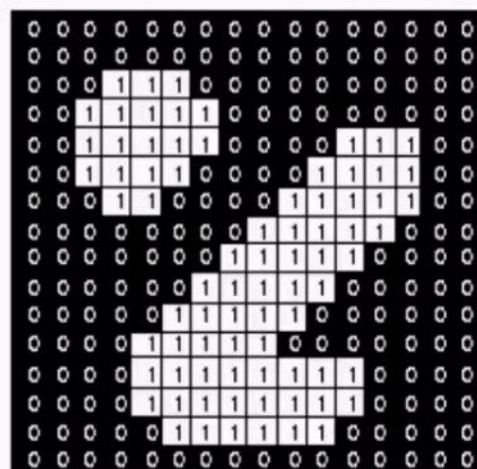
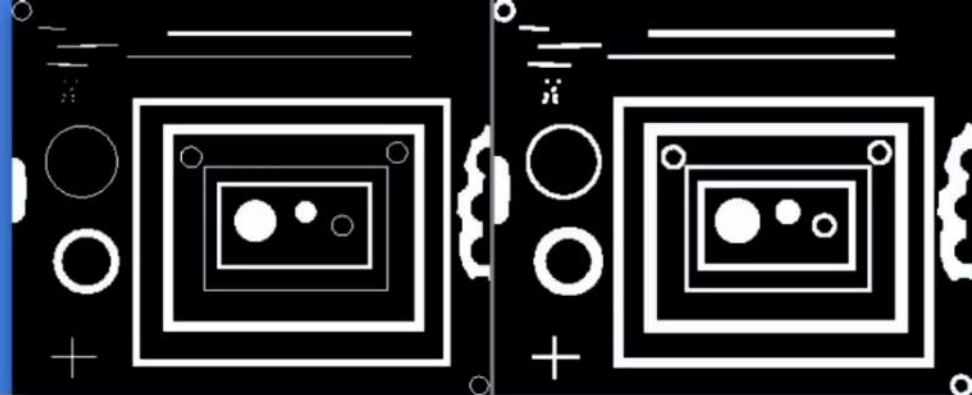
Problema



Dilatación y erosión



Dilatación



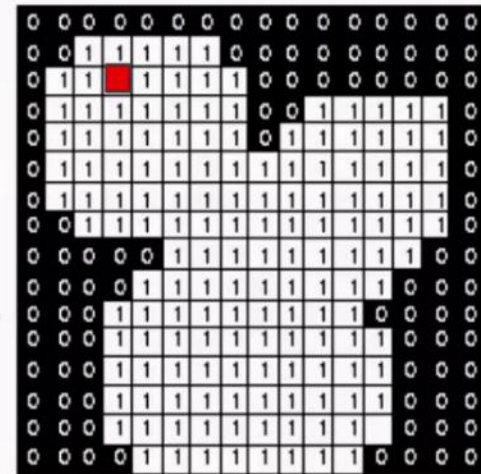
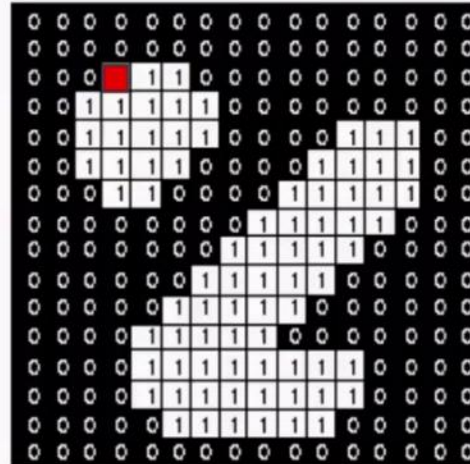
Dilatación paso a paso

- Datos
 - Imagen binaria a analizar
 - Elemento estructural
 - Imagen binaria a generar
- Para cada píxel **true** de la imagen analizada
 - ventana deslizante
 - se posiciona el ancla en la imagen binaria a generar
 - se aplican los valores **true** del **elemento estructural**



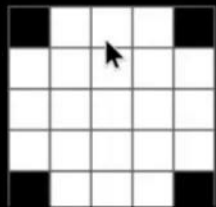
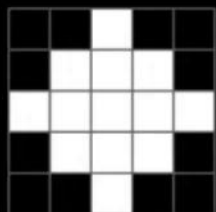
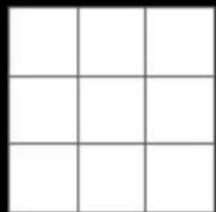
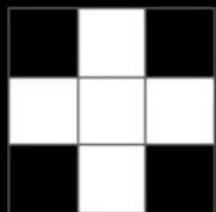
Dilatación paso a paso

- Datos
 - Imagen binaria a analizar
 - Elemento estructural
 - Imagen binaria a generar
- Para cada píxel **true** de la imagen analizada
 - ventana deslizante
 - se posiciona el ancla en la imagen binaria a generar
 - se aplican los valores **true** del **elemento estructural**



Elementos estructurales

- Efecto de diferentes elementos estructurales usados en dilataciones
- Los elementos estructurales más frecuentes para el tratamiento de imágenes en general son
 - cuadrado de 3x3
 - círculo de 5x5 y mayores
- Otros elementos estructurales se usan en efectos específicos



Dilation

Dilation

Dilation

Dilation

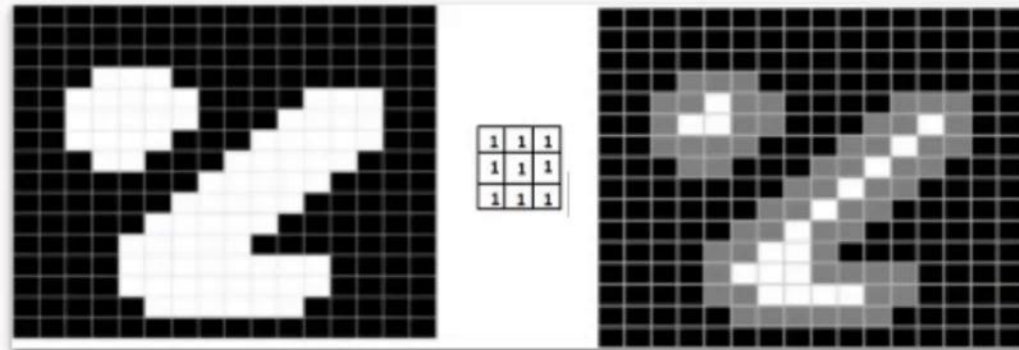
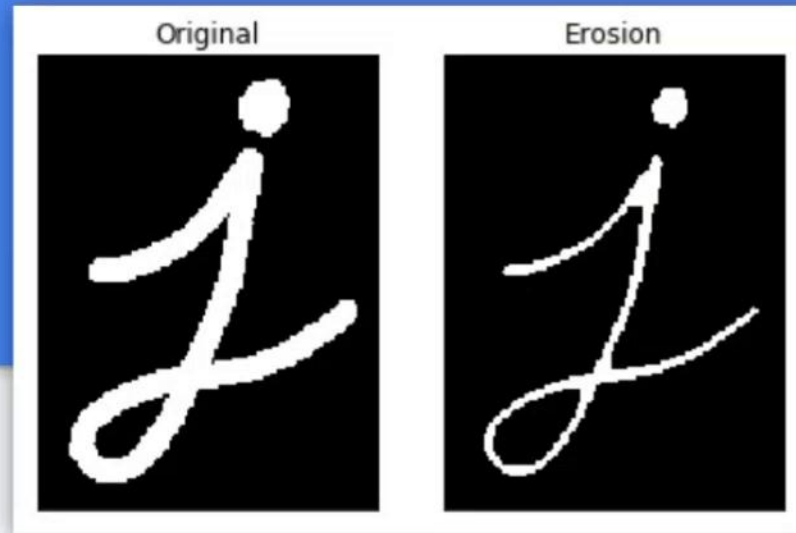
Dilation

Elementos estructurales

- También denominado kernel
- Es una pequeña imagen binaria, con un ancla (usualmente en el centro)
- Predefinidos en OpenCV (paramétricos)
 - MORPH_RECT
 - MORPH_CROSS
 - MORPH_ELLIPSE
- **getStructuringElement()**

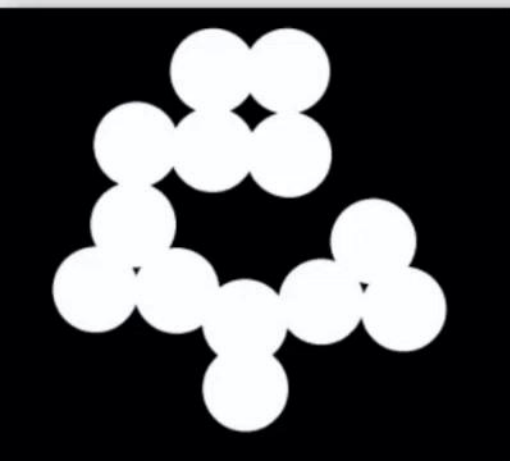
Erosión, versión invertida de la dilatación

- Para cada píxel **false** de la imagen analizada
 - se posiciona el ancla en la imagen binaria a generar
 - en los valores **true** del elemento estructural se aplica **false** a sobre la imagen generada
- Se usa para separar figuras que se tocan en algún punto

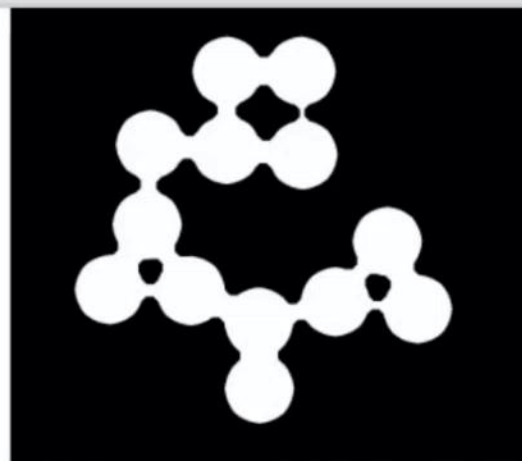




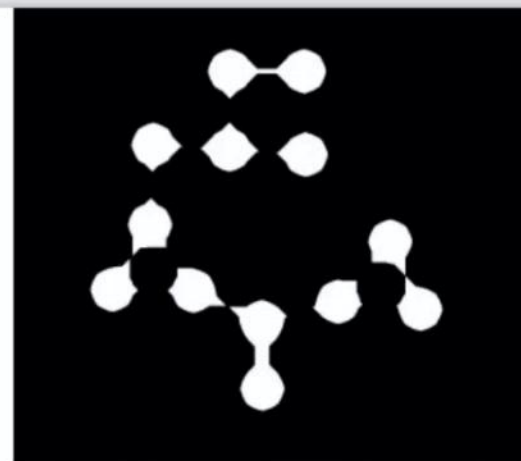
Separación por erosión



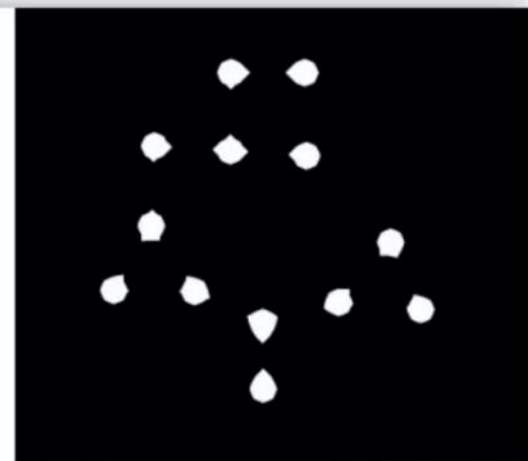
Original binary image
Circles (792x892)



Erosion by disk-shaped
structuring element
Diameter=15



Erosion by disk-shaped
structuring element
Diameter=35



Erosion by disk-shaped
structuring element
Diameter=48

Restauración de imágenes binarias

Open & close en la eliminación del ruido

- Eliminan de ruido
 - remueven elementos pequeños
- Open
 - erosión - dilatación
 - elimina ruido true (puntos blancos)
- Close
 - dilatación - erosión
 - elimina ruido false (puntos negros)



Close: removedor de
agujeros

Top hat & black hat para obtener el “ruido”

- **Top hat**
 - imagen original - opening
 - El resultado son los puntos blancos eliminados por opening
- **Black hat**
 - closing - imagen original
 - El resultado son los puntos negros eliminados por closing, pero blancos



Gradiente

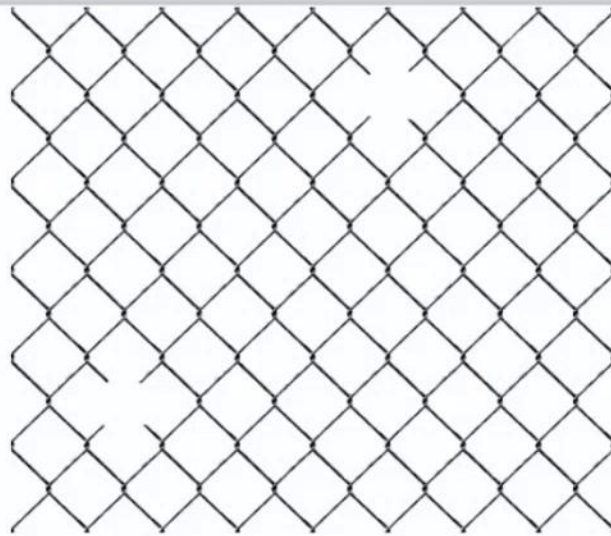
- La manera morfológica de obtener bordes
- Gradiente
 - dilatación - erosión
- Si la imagen tiene ruido, se magnifica



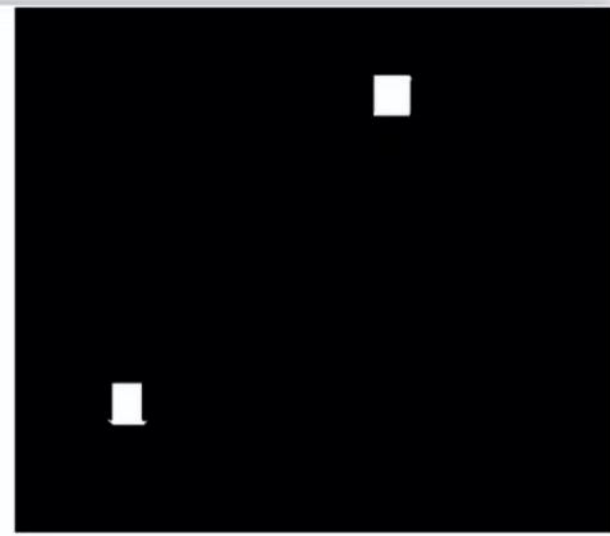
Detección por erosión



Original grayscale image
Fence (1023 x 1173)



Fence thresholded
using Otsu's method



Erosion with 151x151
"cross" structuring element

Hit-miss

- Es una operación morfológica más avanzada
- El elemento estructural tiene valores trinaros 0, 1 y -1
- Produce un true (un hit) cuando
 - Los 1 coinciden con true
 - Los -1 coinciden con false
 - Ignora los 0
- Se usa para marcar píxeles donde se encontraron patrones específicos

Hit-miss

- Es una operación morfológica más avanzada
- El elemento estructural tiene valores trinararios 0, 1 y -1
- Produce un true (un hit) cuando
 - Los 1 coinciden con true
 - Los -1 coinciden con false
 - Ignora los 0
- Se usa para marcar píxeles donde se encontraron patrones específicos

INTEREST-POINT DETECTION

Feature extraction typically starts by finding the salient interest points in the image. For robust image matching, we desire interest points to be repeatable under perspective transformations (or, at least, scale changes, rotation, and translation) and real-world lighting variations. An example of feature extraction is illustrated in Figure 3. To achieve scale invariance, interest points are typically computed at multiple scales using an image pyramid [15]. To achieve rotation invariance, the patch around each interest point is canonically oriented in the direction of the dominant gradient. Illumination changes are compensated by normalizing the mean and standard deviation of the pixels of the gray values within each patch [16].

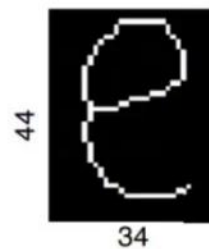
Structuring
element W



INTEREST-POINT DETECTION

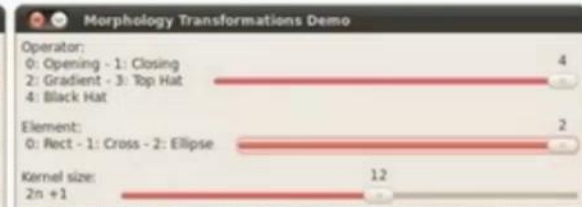
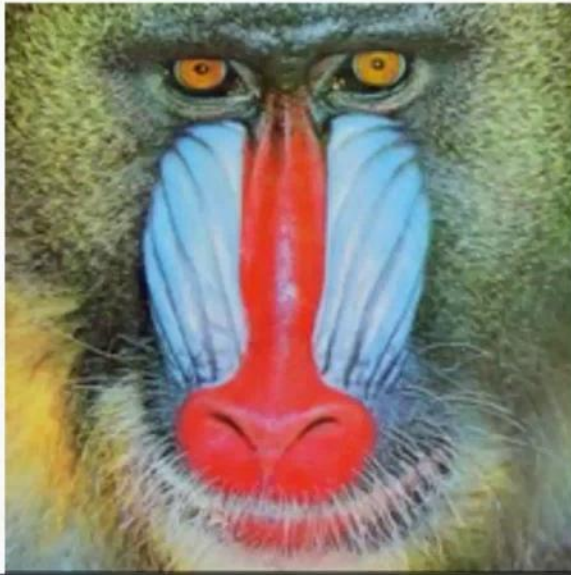
Feature extraction typically starts by finding the salient interest points in the image. For robust image matching, we desire interest points to be repeatable under perspective transformations (or, at least, scale changes, rotation, and translation) and real-world lighting variations. An example of feature extraction is illustrated in Figure 3. To achieve scale invariance, interest points are typically computed at multiple scales using an image pyramid [15]. To achieve rotation invariance, the patch around each interest point is canonically oriented in the direction of the dominant gradient. Illumination changes are compensated by normalizing the mean and standard deviation of the pixels of the gray values within each patch [16].

Structuring
element W

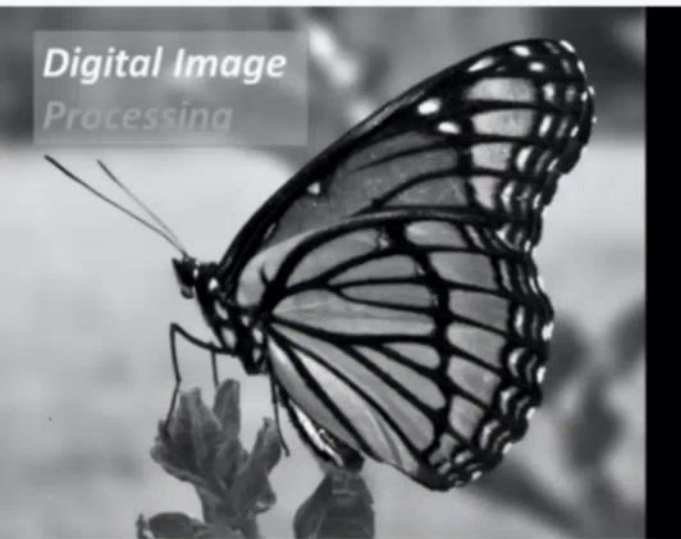


A colores

Tiene usos en procesamiento de imágenes, logrando algunos efectos



Dilatación y erosión en imágenes grises



Original 394 x 305



Dilation 10x10 square



Erosion 10x10 square

getStructuringElement()

- **getStructuringElement()**
 - Tipos
 - MORPH_RECT
 - MORPH_CROSS
 - MORPH_ELLIPSE
 - Se especifica un rectángulo, que define el tamaño del elemento estructural

Prácticas



MILITELLO NICOLAS URIEL ✂



LOPEZ FERME NAHUEL EZEQUIEL ✂



Jorge Alejandro Silvestri



DI STILIO GASTON ✂



PAGNOTTA PABLO ✂



MATEO FEDERICO ✂



ROSANO MATIAS AGUSTIN ✂



Alesio Esteban Sinopoli ✂



SAINI LUIS ALBERTO ✂

